

Systems for AI Basics

CSE 585 Advanced Scalable Systems for Agentic AI

Jan 15, 2026

Shiqi He

Adapted from materials originally created by
Jae-Won Chung

Shiqi He

- Bio
 - 3rd year PhD student working with Mosharaf
 - <https://tctower.github.io/>
- Background
 - 2020 – 2023: Efficient Machine Learning, Federated Learning
 - 2023 – now: Agentic AI Systems, Efficient Multimodal Models, Diffusion Serving
- Three lectures
 - Jan 15 (Thu) - Systems for AI Basics
 - Jan 20 (Tue) - Distributed Training Basics
 - Feb 10 (Tue) - Inference Basics

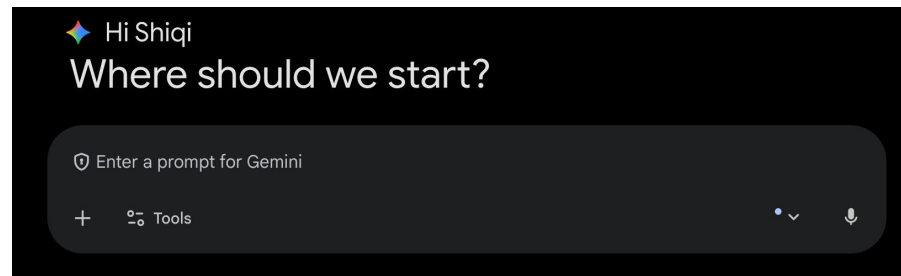
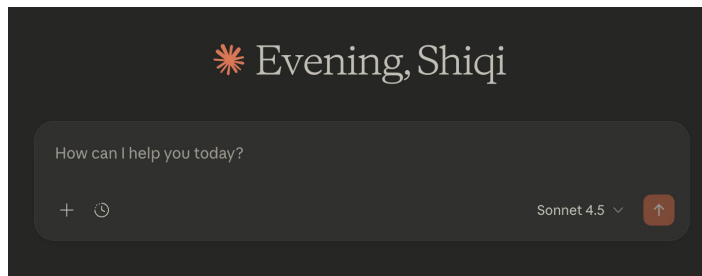
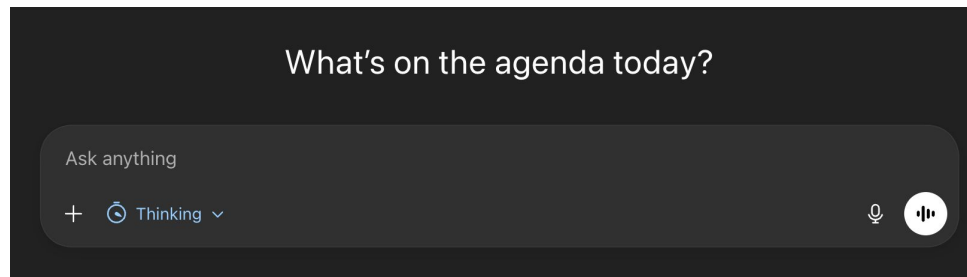
Agenda

- Overview of GenAI
 - From Transformers to LLMs
 - Multi-modal Models (Any-to-Text)
 - Diffusion Models (Text-to-Any)
 - LLM AI Agent
- Overview of Systems for GenAI
 - A high-level landscape of systems challenges
- Please ask questions if you don't get something

Agenda

- Overview of GenAI
 - **From Transformers to LLMs**
 - Multi-modal Models (Any-to-Text)
 - Diffusion Models (Text-to-Any)
 - LLM AI Agent
- Overview of Systems for GenAI
 - A high-level landscape of systems challenges
- Please ask questions if you don't get something

How did we end up with this?



Attention is All You Need (Google, 2017)

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

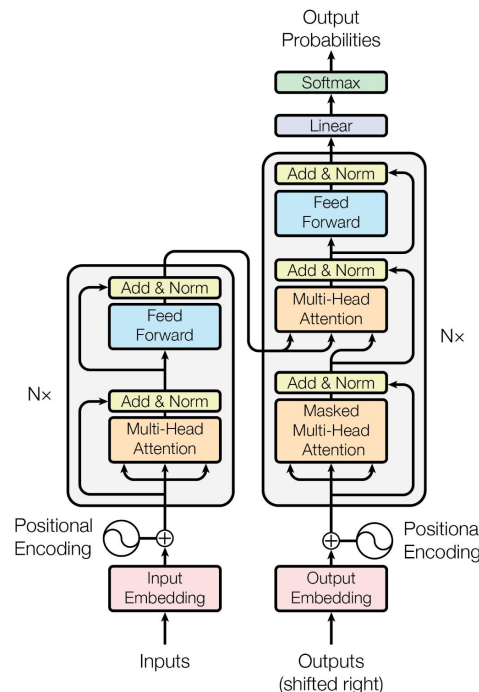
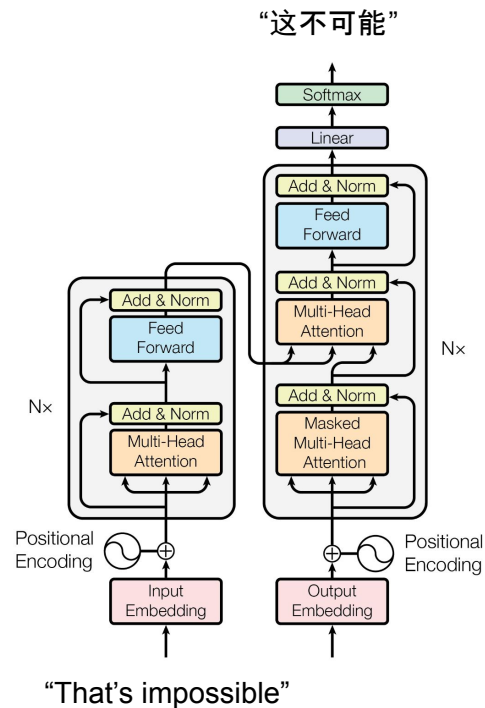
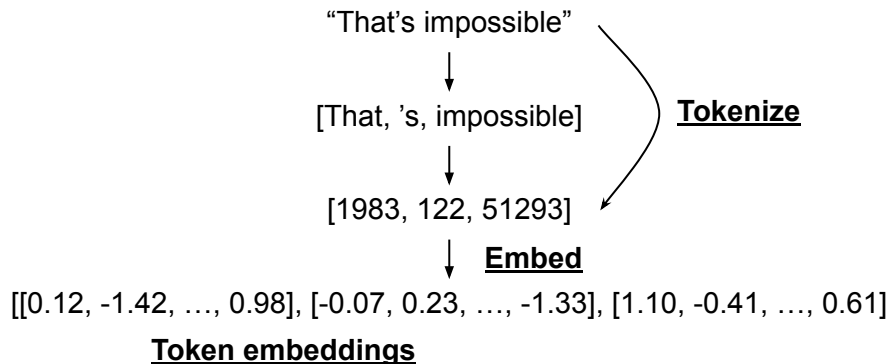


Figure 1: The Transformer - model architecture.

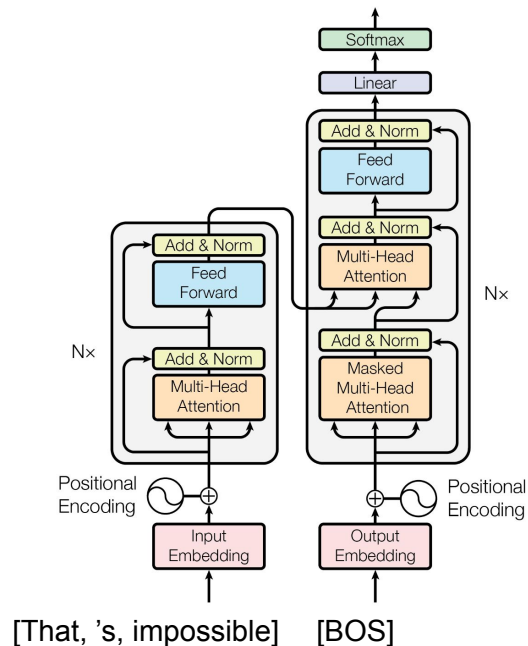
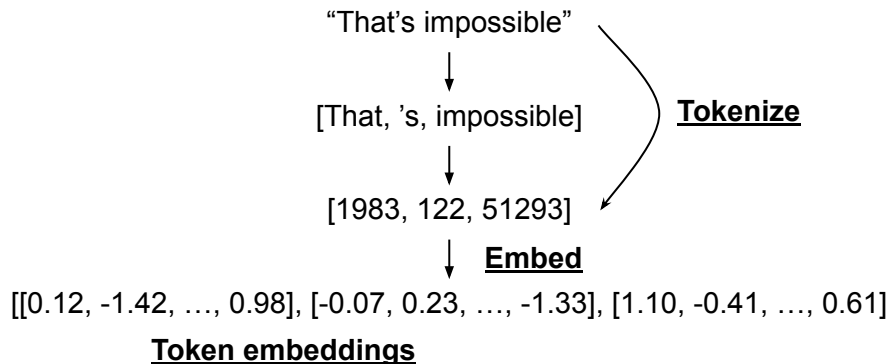
Attention is All You Need (Google, 2017)

- The Transformer model
 - [The Illustrated Transformer](#)
 - Sequence-to-sequence learning
- Critical components
 - Tokenization & embedding
 - Attention
 - MLP



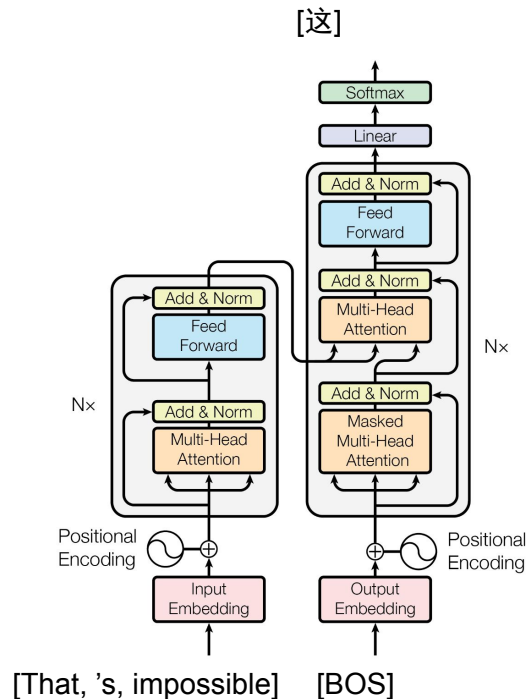
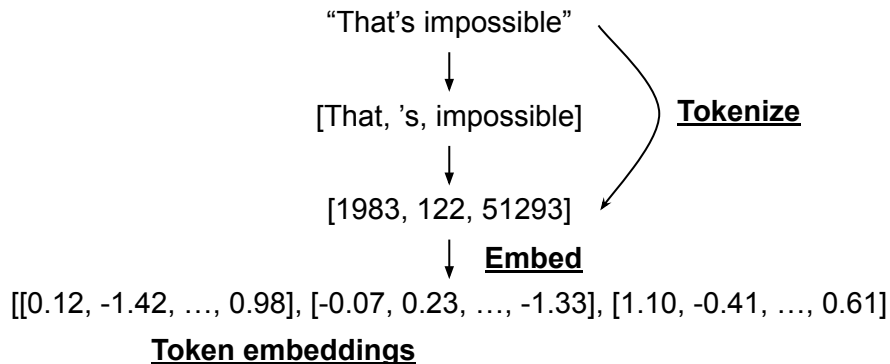
Attention is All You Need (Google, 2017)

- Autoregressive text generation
 - One forward pass produces one output token
 - Keep appending output token to input
 - Stop when the special EOS token is produced



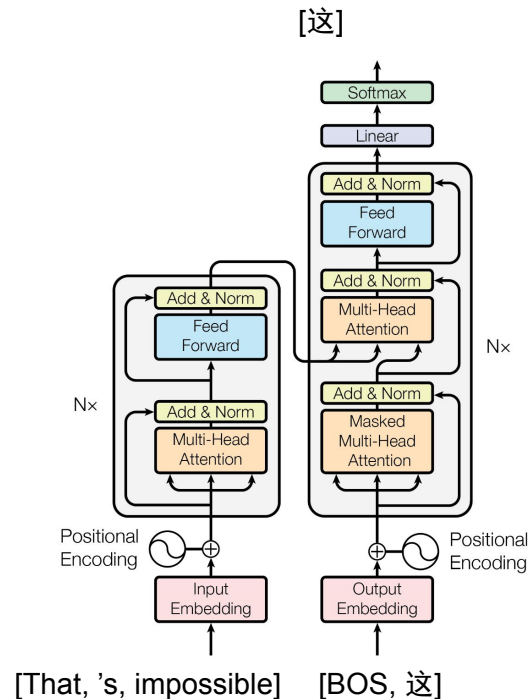
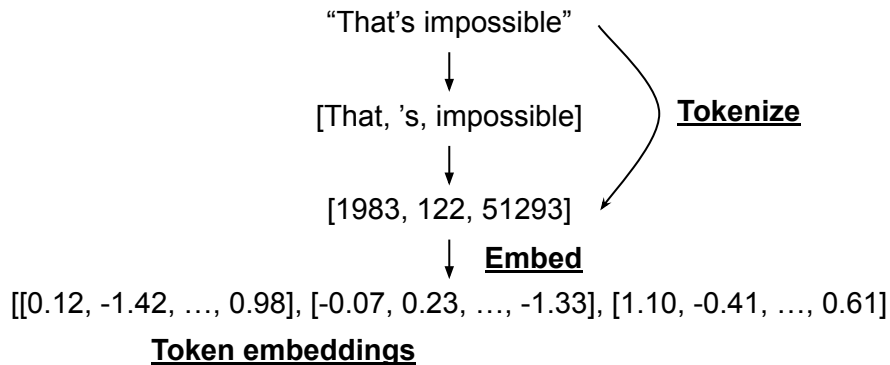
Attention is All You Need (Google, 2017)

- Autoregressive text generation
 - One forward pass produces one output token
 - Keep appending output token to input
 - Stop when the special EOS token is produced



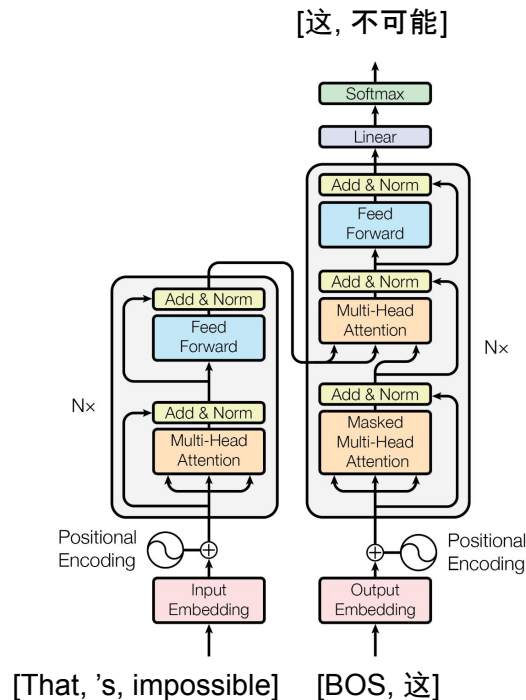
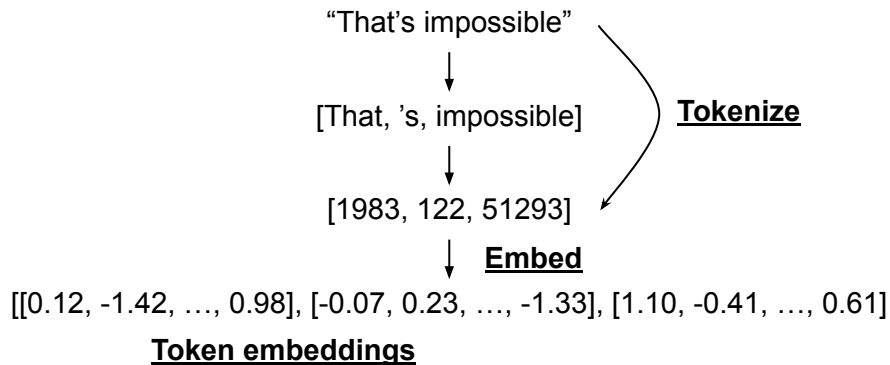
Attention is All You Need (Google, 2017)

- Autoregressive text generation
 - One forward pass produces one output token
 - Keep appending output token to input
 - Stop when the special EOS token is produced



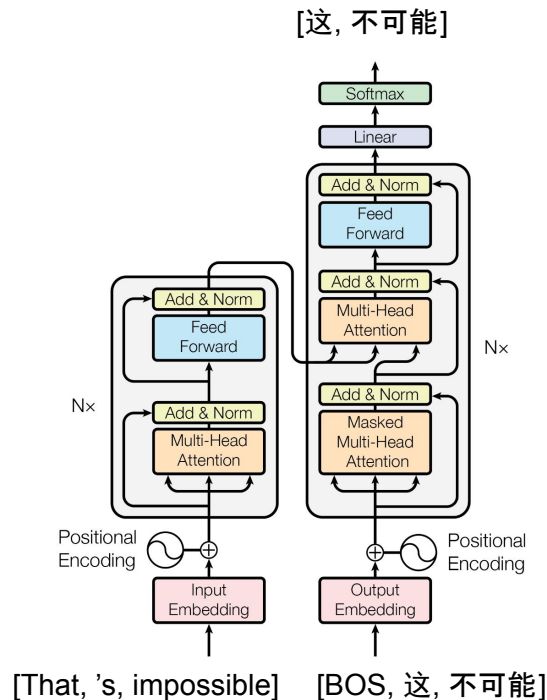
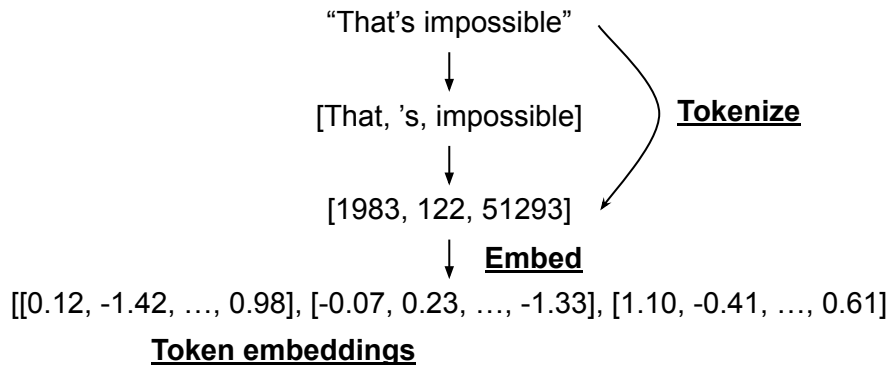
Attention is All You Need (Google, 2017)

- Autoregressive text generation
 - One forward pass produces one output token
 - Keep appending output token to input
 - Stop when the special EOS token is produced



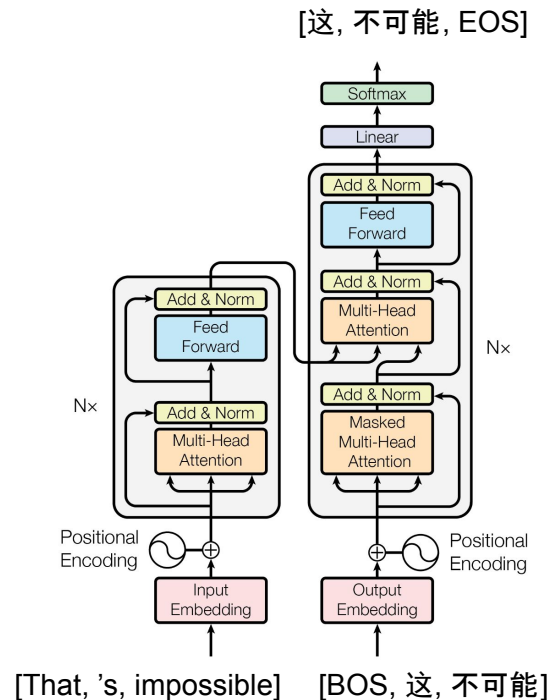
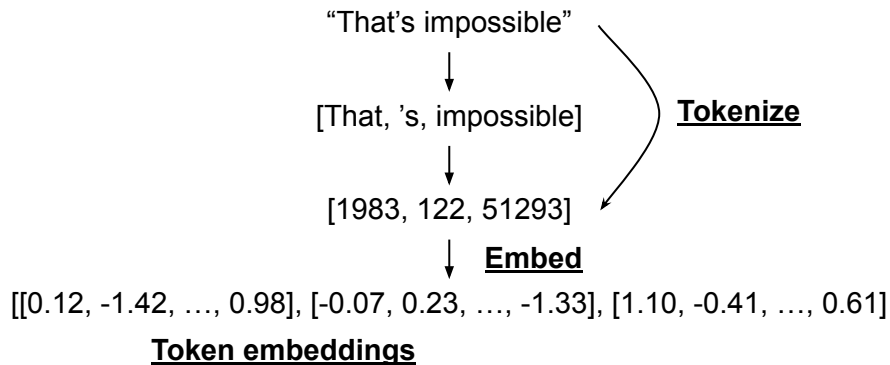
Attention is All You Need (Google, 2017)

- Autoregressive text generation
 - One forward pass produces one output token
 - Keep appending output token to input
 - Stop when the special EOS token is produced



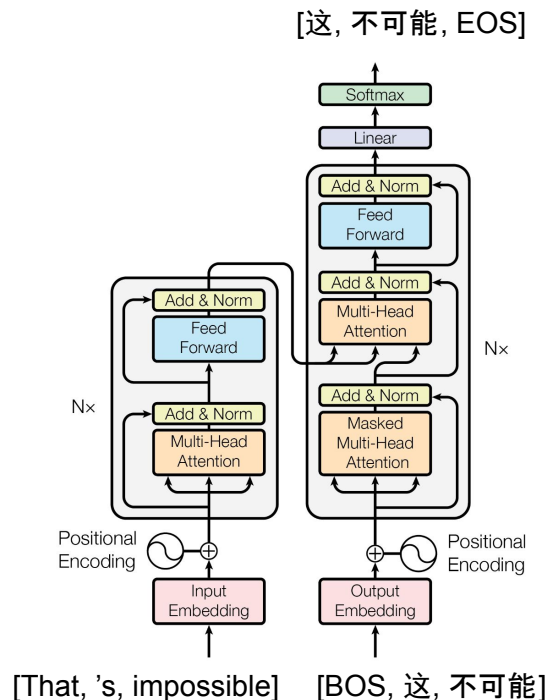
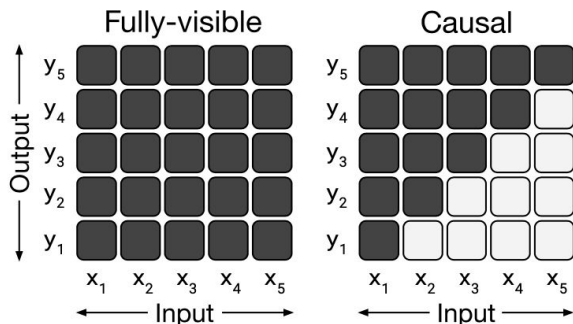
Attention is All You Need (Google, 2017)

- Autoregressive text generation
 - One forward pass produces one output token
 - Keep appending output token to input
 - Stop when the special EOS token is produced



Attention is All You Need (Google, 2017)

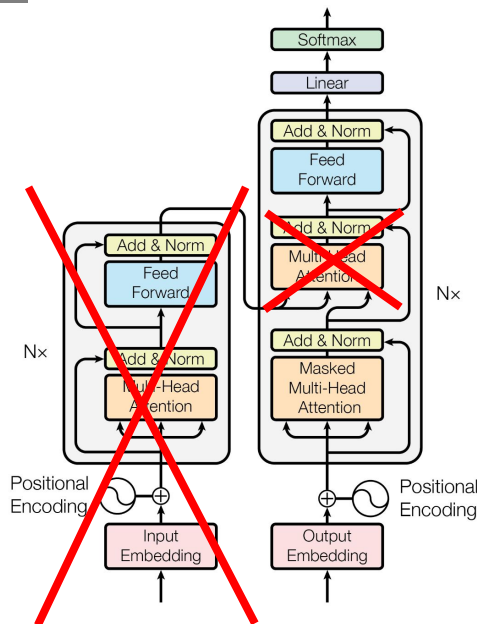
- The **Attention** mechanism
 - Details deferred to [The Illustrated Transformer](#)
 - Importance-weighted sum over token embeddings
 - Two types
 - **Causal attention (decoder)**
 - Token i aggregates over *previous* tokens
 - Bidirectional attention (encoder)
 - Token i aggregates over *every* token



GPT-1 (OpenAI, 2018)

- Generative Pre-trained Transformer

- [Improving language understanding with unsupervised learning](#) (OpenAI blog post)
- An autoregressive text generation model
- 117M parameters, 5GB of text



Decoder-only Transformer

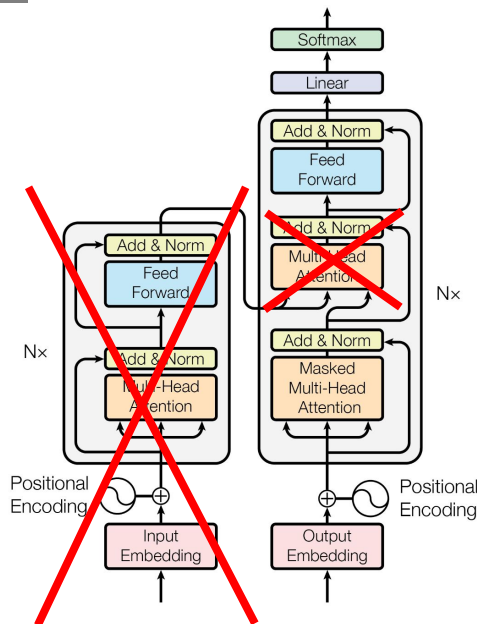
GPT-1 (OpenAI, 2018)

- Generative Pre-trained Transformer

- [Improving language understanding with unsupervised learning](#) (OpenAI blog post)
- An autoregressive text generation model
- 117M parameters, 5GB of text

- Benefits

- Multi-turn generation
- Next-token prediction
- Simple, cheaper
- Scale better



Decoder-only Transformer

GPT-1 (OpenAI, 2018)

- Generative Pre-trained Transformer

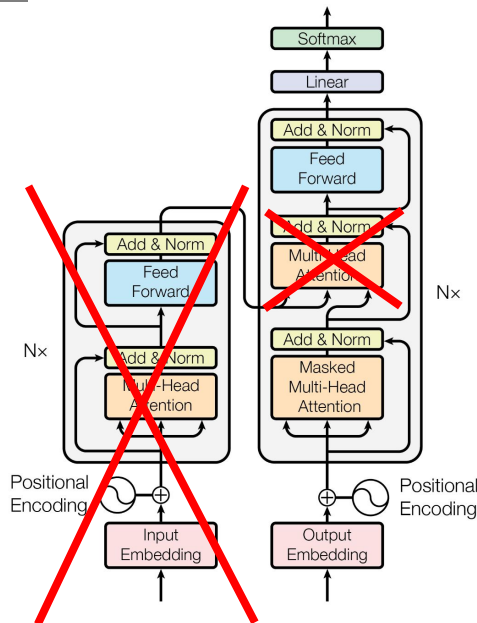
- [Improving language understanding with unsupervised learning](#) (OpenAI blog post)
- An autoregressive text generation model
- 117M parameters, 5GB of text

- Benefits

- Multi-turn generation
- Next-token prediction
- Simple, cheaper
- Scale better

- Encoder-decoder still has its Advantages

- Long context understanding



Decoder-only Transformer

GPT (OpenAI, 2018)

- A new paradigm
 - First, **pre-train** a large model on a large unlabeled corpus of text
 - Then, **fine-tune** the model on a *small* labeled task-specific dataset (e.g., translation)
 - The traditional way was to figure out how to build a *large* task dataset and train on it
 - Now, you can pre-train a large model once and reuse it for many downstream tasks

Next Token Prediction

- Attention is all you → ?
- Large language → ?
- University of California, San → ?
- This is the best steak I've ever had in my entire → ?

GPT (OpenAI, 2018)

- A new paradigm
 - First, **pre-train** a large model on a large unlabeled corpus of text
 - Then, **fine-tune** the model on a small labeled task-specific dataset (e.g., translation)
- Systems challenge?

Drawbacks

This project has a few outstanding issues which are worth noting:

- **Compute Requirements:** Many previous approaches to NLP tasks train relatively small models on a single GPU from scratch. Our approach requires an expensive pre-training step—**1 month on 8 GPUs**. Luckily, this only has to be done once and we're releasing our model so others can avoid it. It is also a large model (in comparison to prior work) and consequently uses more compute and memory—we used a 37-layer (12 block) Transformer architecture, and we train on sequences of up to 512 tokens. Most experiments were conducted on 4 and 8 GPU systems. The model does fine-tune to new tasks very quickly which helps mitigate the additional resource requirements.

5,760 GPU hours!

GPT-2 (OpenAI, 2019)

- Successor to GPT
 - [Better language models and their implications](#) (OpenAI blog)
 - 1.5B parameters (12.8x), 40GB of Internet text (8x)

System Prompt
(human-written)

In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Model Completion
(machine-written,
10 tries)

The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science.

Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved.

Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

GPT-2 (OpenAI, 2019)

- Safety concerns and the closed model strategy

Release strategy

Due to concerns about large language models being used to generate deceptive, biased, or abusive language at scale, we are only releasing a much smaller version of GPT-2 along with sampling code. We are not releasing the dataset, training code, or GPT-2 model weights. Nearly a year ago we wrote in the OpenAI Charter: “we expect that safety and security concerns will reduce our traditional publishing in the future, while increasing the importance of sharing safety, policy, and standards research,” and we see this current work as potentially representing the early beginnings of such concerns, which we expect may grow over time. This decision, as well as our discussion of it, is an experiment: while we are not sure that it is the right decision today, we believe that the AI community will eventually need to tackle the issue of publication norms in a thoughtful way in certain research areas. Other disciplines such as biotechnology and cybersecurity have long had active debates about responsible publication in cases with clear misuse potential, and we hope that our experiment will serve as a case study for more nuanced discussions of model and code release decisions in the AI community.

Scaling Laws (OpenAI, Kaplan et al., Jan 2020)

- GPT-1 to GPT-2
 - Model parameters: 117M $\rightarrow$ 1.5B (12.8x)
 - Data: 5 GB $\rightarrow$ 40 GB (8x)
- Questions
 - Should we continue scaling the number of parameters and the dataset?
 - What's the best ratio between scaling model parameters and the dataset?
 - Equivalently, given the same amount of target compute, what's the (#params, data size) pair that leads to the best model quality?

Scaling Laws (OpenAI, Kaplan et al., Jan 2020)

- Questions
 - Should we continue scaling the number of parameters and the dataset?
 - What's the best ratio between scaling model parameters and the dataset?
- “Let’s do a grid search and see what happens.”
 - [Scaling Laws for Neural Language Models](#) (OpenAI)

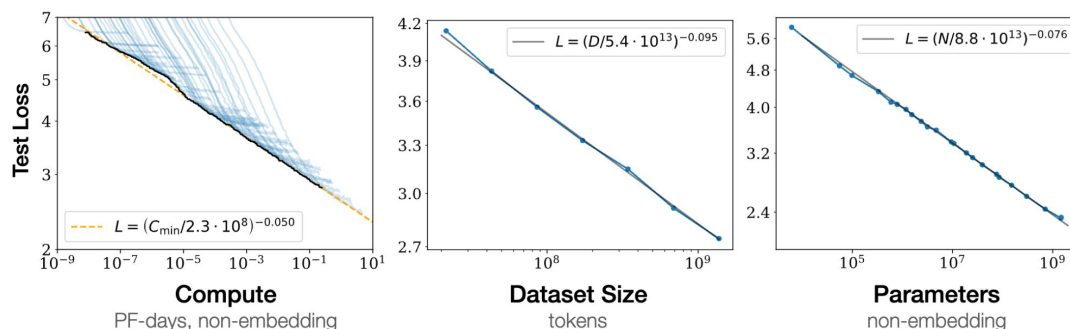


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Scaling Laws (OpenAI, Kaplan et al., Jan 2020)

- Questions *and* Kaplan et al.'s answer
 - Should we continue scaling the number of parameters and the dataset? *Yeah, probably.*
 - What's the best ratio between scaling model parameters and the dataset? *Roughly equally.*
- “Let’s do a grid search and see what happens.”
 - [Scaling Laws for Neural Language Models](#) (OpenAI)

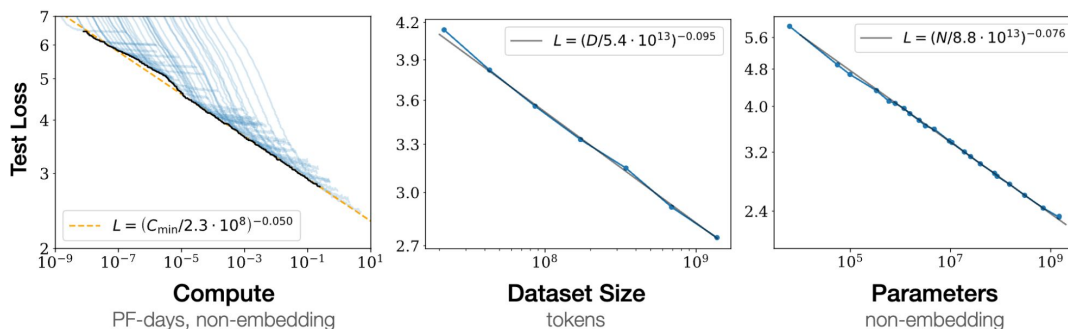


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Scaling Laws (OpenAI, Jan 2020)

- Two notable progressions after this
 - [Chinchilla scaling law](#) (Google DeepMind, Hoffman et al., 2022)
 - You should actually scale data more!

Scaling Laws (OpenAI, Jan 2020)

- Two notable progressions after this
 - [Chinchilla scaling law](#) (Google DeepMind, Hoffman et al., 2022)
 - You should actually scale data more!
 - Overtraining: Train models on more data than the Chinchilla-optimal point
 - Your training time compute is no longer *optimal* (in Chinchilla sense)
 - You end up with a smaller model with good model quality
 - But smaller models are cheaper to run inference on!

LLaMA: Open and Efficient Foundation Language Models

Hugo Touvron*, Thibaut Lavril*, Gautier Izacard*, Xavier Martinet
Marie-Anne Lachaux, Timothee Lacroix, Baptiste Rozière, Naman Goyal
Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin
Edouard Grave*, Guillaume Lample*

Meta AI

The objective of the scaling laws from [Hoffmann et al. \(2022\)](#) is to determine how to best scale the dataset and model sizes for a particular *training* compute budget. However, **this objective disregards the inference budget**, which becomes critical when serving a language model at scale. In this context, given a target level of performance, the preferred model is not the fastest to train but the fastest at inference, and although it may be cheaper to train a large model to reach a certain level of performance, a smaller one trained longer will ultimately be **cheaper at inference**. For instance, although [Hoffmann et al. \(2022\)](#) recommends training a 10B model on 200B tokens, we find that the performance of a 7B model continues to improve even after 1T tokens.

GPT-3 (OpenAI, May 2020)

- GPT-3
 - [Language Models are Few-Shot Learners](#) (OpenAI paper)
 - 175B parameters, 400B tokens
- A new paradigm
 - Previously: Pre-train on large corpus, and then fine-tune on task-specific dataset.
 - GPT-3: Don't even do the fine-tuning part. Instead, put a **few** examples in the prompt.

Few-shot examples

Actual prompt |
GPT's completion |

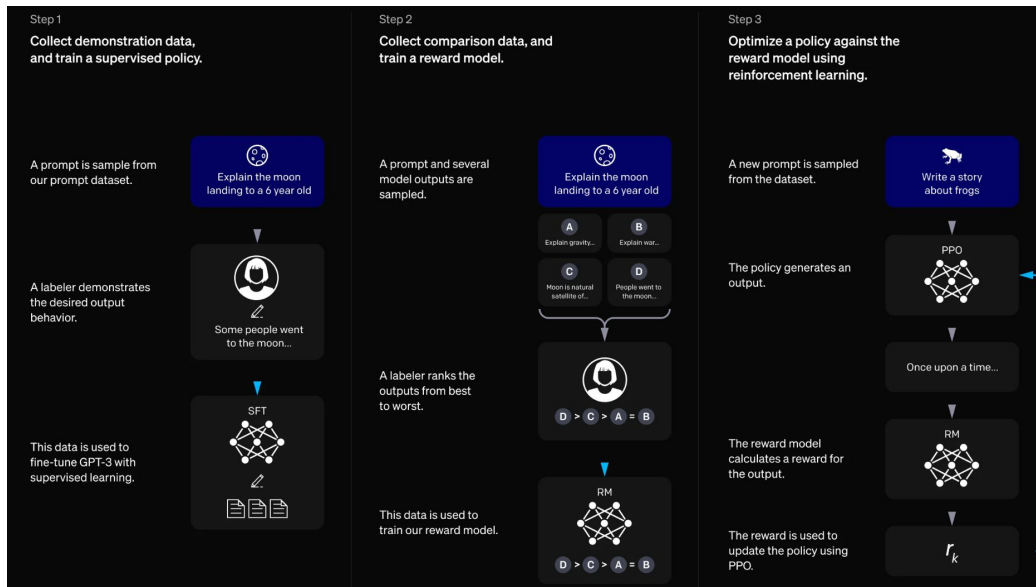
```
Poor English input:  I eated the purple berries.
Good English output: I ate the purple berries.
Poor English input:  Thank you for picking me as your designer.  I'd appreciate it.
Good English output: Thank you for choosing me as your designer.  I appreciate it.
Poor English input:  The mentioned changes have done.  or I did the alteration that you
requested.  or I changed things you wanted and did the modifications.
Good English output: The requested changes have been made.  or I made the alteration that you
requested.  or I changed things you wanted and made the modifications.
Poor English input:  I'd be more than happy to work with you in another project.
Good English output: I'd be more than happy to work with you on another project.
```

RLHF (OpenAI, Jan 2022)

- Reinforcement Learning with Human Feedback (RHLF)
 - [Aligning language models to follow instructions](#) (OpenAI blog)
- Few-shot prompting works, but it has limitations
 - Hope: We want GPT to do text completion in a super friendly way
 - But nothing in the pre-training corpus really endorses the model to be friendly
 - We do need a step after pre-training to reinforce certain behaviors

RLHF (OpenAI, Jan 2022)

- Three steps
 - Among many GPT responses to the same prompt, a human indicates which one they prefer
 - Train a *reward model* that predicts human preference for a given response
 - Run RL on the LLM using the reward model (**PPO**; Proximal Policy Optimization)



ChatGPT (OpenAI, Nov 2022)

- Still a text generation model that predicts the next token!
- But the prompt is formatted in a particular way

Real prompt rendering of the OpenAI GPT OSS 20B model.
The **template** is baked into the model during **post-training**.

System prompt: You are a helpful assistant.
User's prompt: Write a poem about systems.

**Prompt
rendering**

```
<|start|>system<|message|>You are ChatGPT, a  
large language model trained by OpenAI.  
Knowledge cutoff: 2024-06  
Current date: 2025-08-28  
Reasoning: medium  
  
# Valid channels: analysis, commentary, final.  
Channel must be included for every  
message.<|end|><|start|>developer<|message|>#  
Instructions  
  
You are a helpful assistant.  
  
<|end|><|start|>user<|message|>Write a poem  
about systems.<|end|><|start|>assistant
```

ChatGPT (OpenAI, Nov 2022)

- Still a text generation model that predicts the next token!
- But the prompt is formatted in a particular way

Real prompt rendering of the OpenAI GPT OSS 20B model.
The **template** is baked into the model during **post-training**.

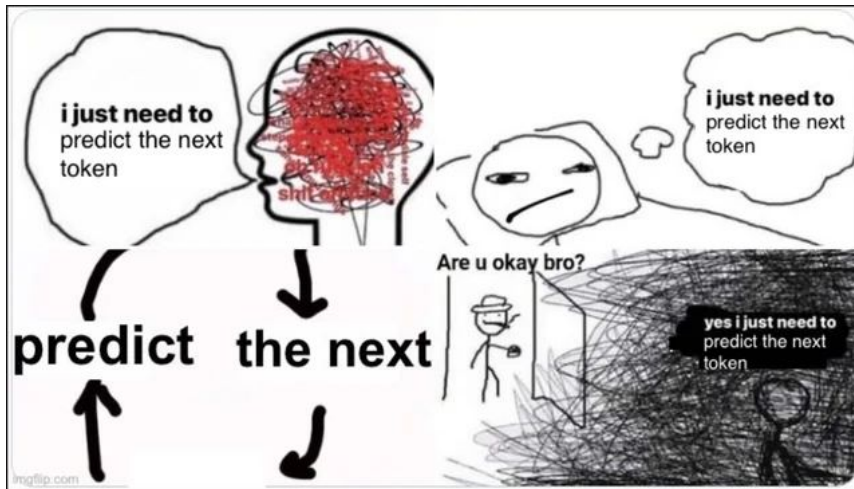
System prompt: You are a helpful assistant.
User's prompt: Write a poem about systems.

**Prompt
rendering**

```
<|start|>system<|message|>You are ChatGPT, a  
large language model trained by OpenAI.  
Knowledge cutoff: 2024-06  
Current date: 2025-08-28  
Reasoning: medium  
  
# Valid channels: analysis, commentary, final.  
Channel must be included for every  
message.<|end|><|start|>developer<|message|>#  
Instructions  
  
You are a helpful assistant.  
  
<|end|><|start|>user<|message|>Write a poem  
about systems.<|end|><|start|>assistant
```

ChatGPT (OpenAI, Nov 2022)

- Still a text generation model that predicts the next token!
- But the prompt is formatted in a particular way



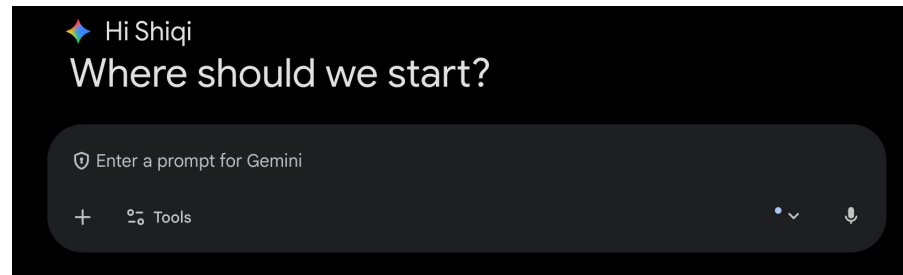
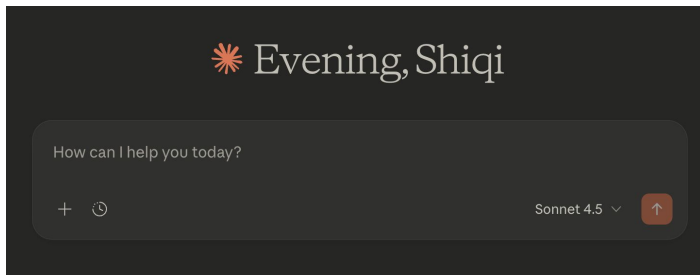
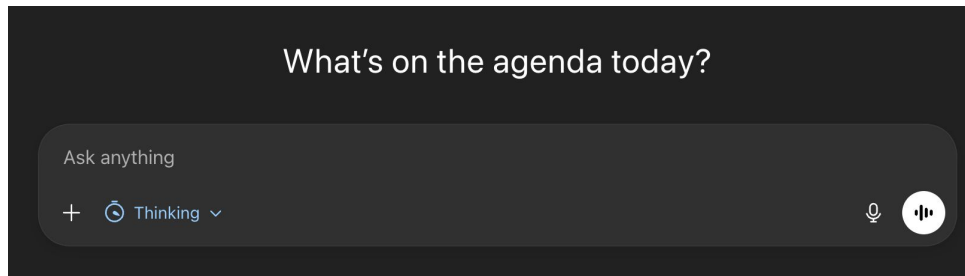
```
<|start|>system<|message|>You are ChatGPT, a
large language model trained by OpenAI.
Knowledge cutoff: 2024-06
Current date: 2025-08-28
Reasoning: medium
```

```
# Valid channels: analysis, commentary, final.
Channel must be included for every
message.<|end|><|start|>developer<|message|>#
Instructions
```

```
You are a helpful assistant.
```

```
<|end|><|start|>user<|message|>Write a poem
about systems.<|end|><|start|>assistant
```

ChatGPT (OpenAI, Nov 2022)

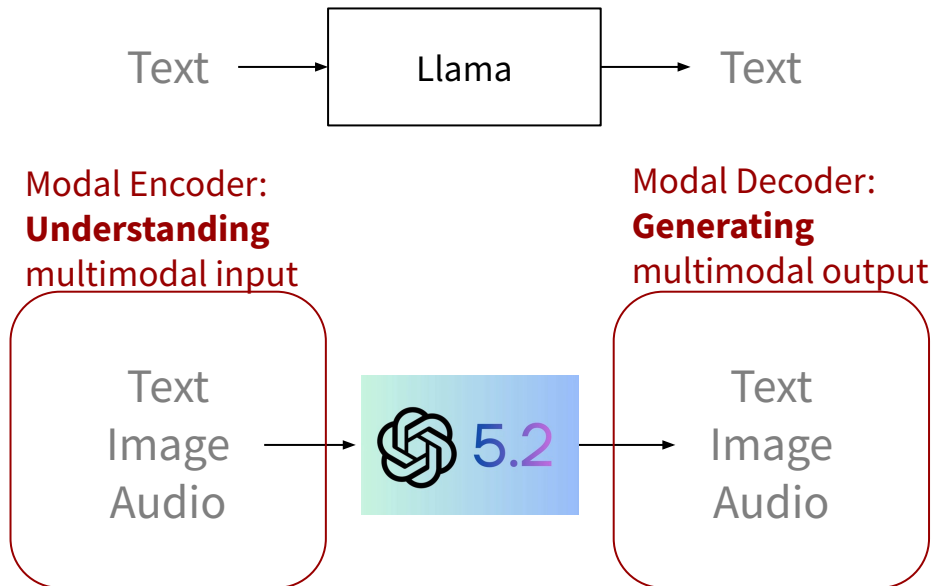


Agenda

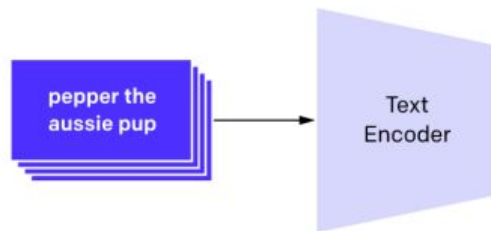
- Overview of GenAI
 - From Transformers to LLMs
 - **Multi-modal Models (Any-to-Text)**
 - Diffusion Models (Text-to-Any)
 - LLM AI Agent
- Overview of Systems for GenAI
 - A high-level landscape of systems challenges
- Please ask questions if you don't get something

Multi-Modalities: Towards Human-like AI

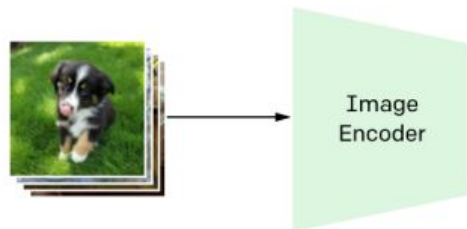
- Human intelligence: uses more types of data than just text
- Modality: ways in which we perceive and express information



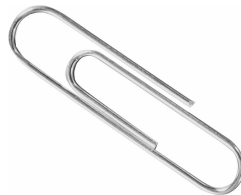
How to Connect Different Modalities?



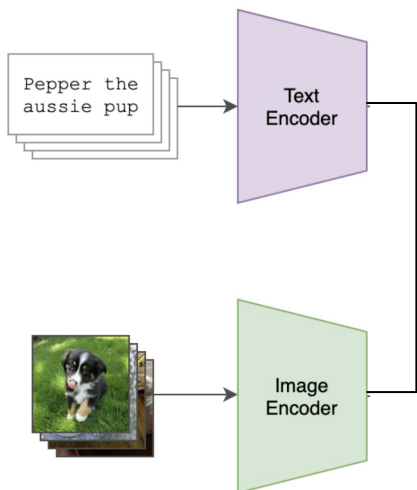
?



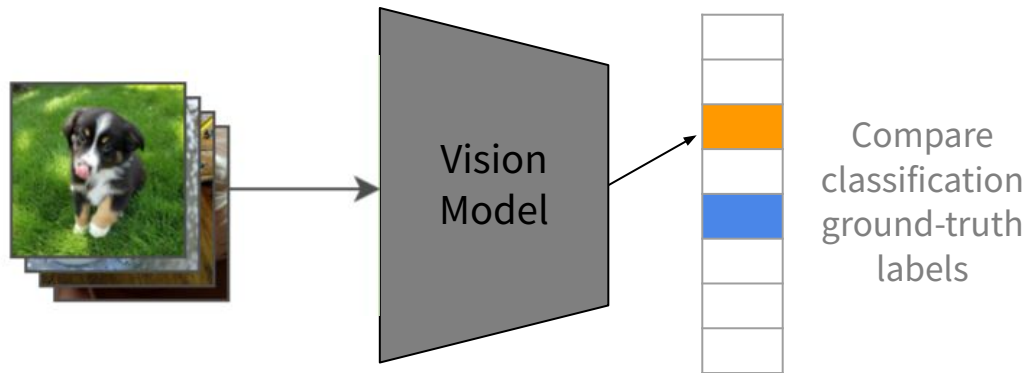
All Began with... CLIP



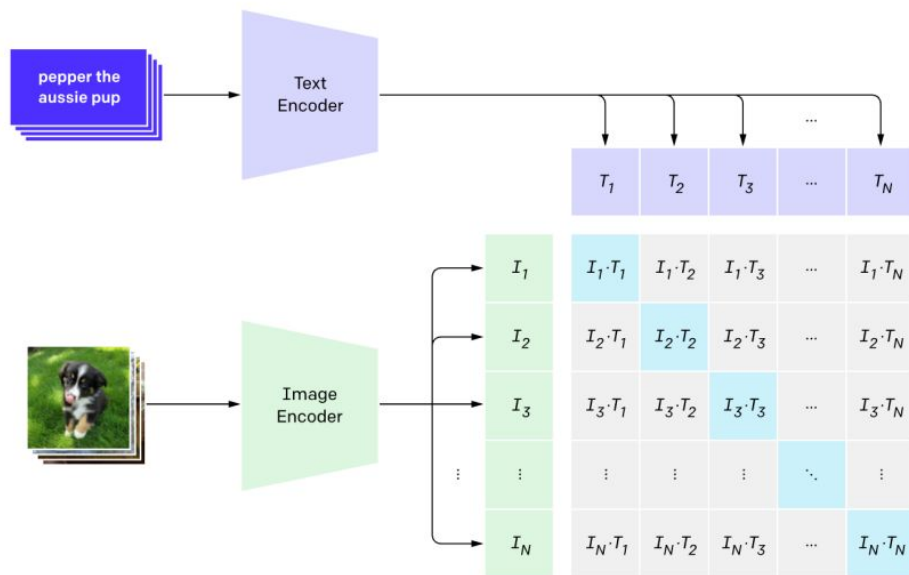
- OpenAI CLIP (ICML21): Contrastive Language-Image Pre-training
 - Connect textual descriptions with images



vs. Simple Vision Model Training (classification)

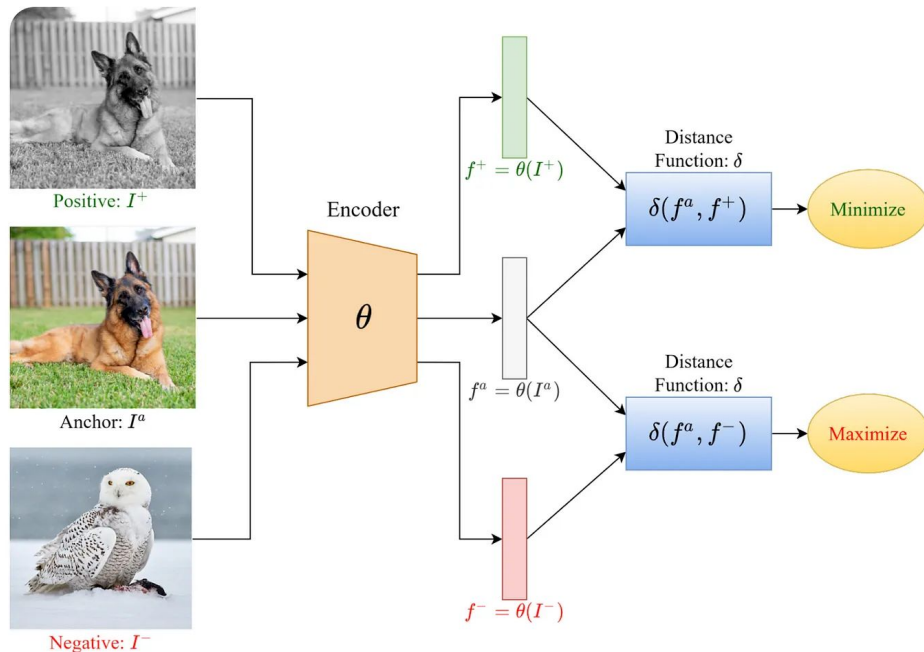


CLIP Overview - Training



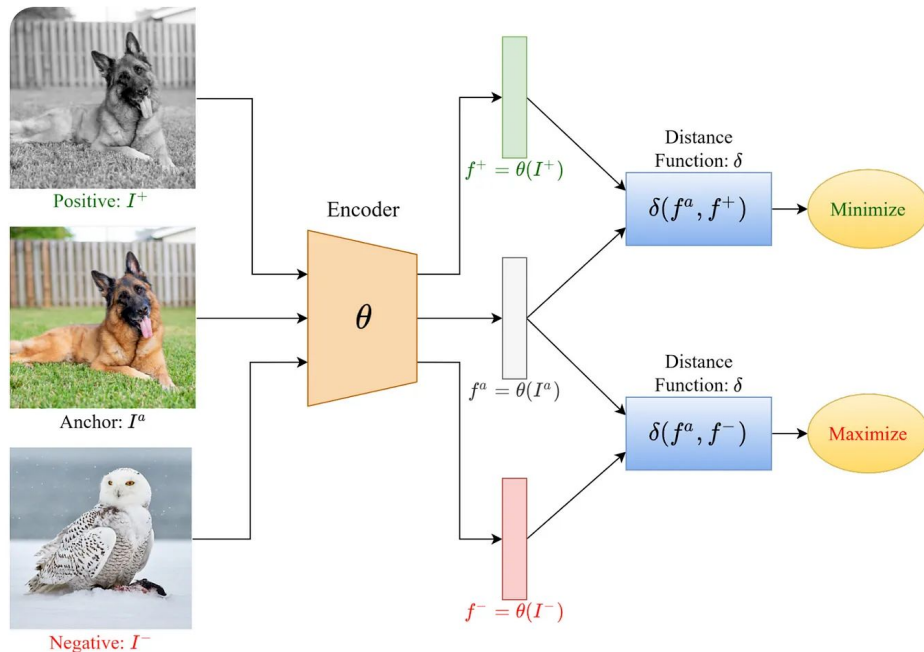
- Learn the relationship between textual description and image
 - Positive score for its own label
 - Negative score for all the others

CLIP Overview - Training

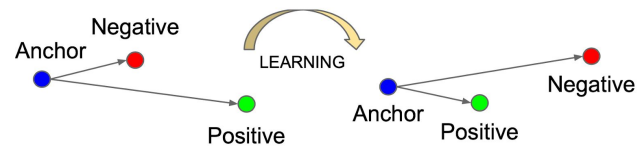


- Use (Anchor, positive, negative) set to train the model
 - Anchor: an image from one class
 - Positive: another image in the same class with the anchor image
 - Negative: an image from different class

CLIP Overview - Training



- Use (Anchor, positive, negative) set to train the model
 - Anchor: an image from one class
 - Positive: another image in the same class with the anchor image
 - Negative: an image from different class
- Minimize distance (Anchor, Pos)
Maximize distance (Anchor, Neg)



CLIP Limitations

(i)



(ii)



(iii)



Images

Caption1: Soccer winger and football player vie for the ball during semi.

Caption2: Football player vies with football player for the ball during sports association.

Caption3: Football player celebrates after scoring the opening goal with football player during football league.

Captions

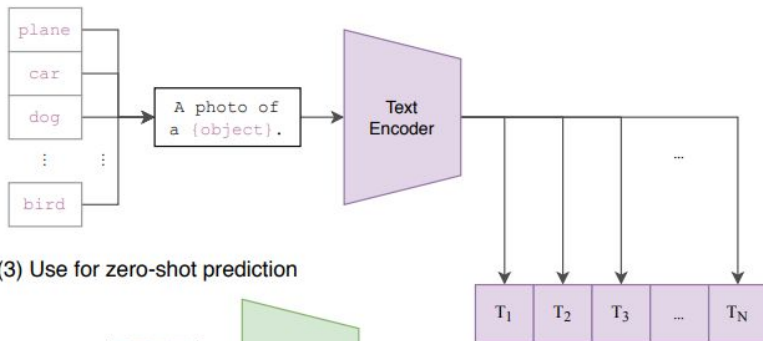
- Is it really correct to mark all the other images simply negative?



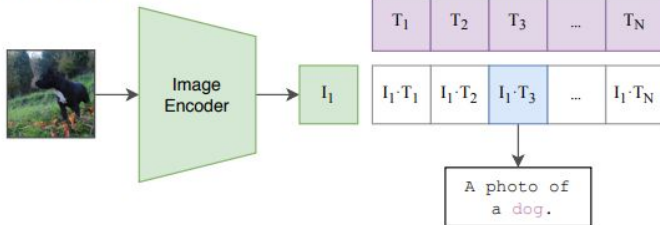
<i>Cap1</i>	1	0	0
<i>Cap2</i>	0	1	0
<i>Cap3</i>	0	0	1

CLIP Overview - Inference

(2) Create dataset classifier from label text



(3) Use for zero-shot prediction



- Select a class with the highest similarity with the image, and fill the given sentence (zero-shot)

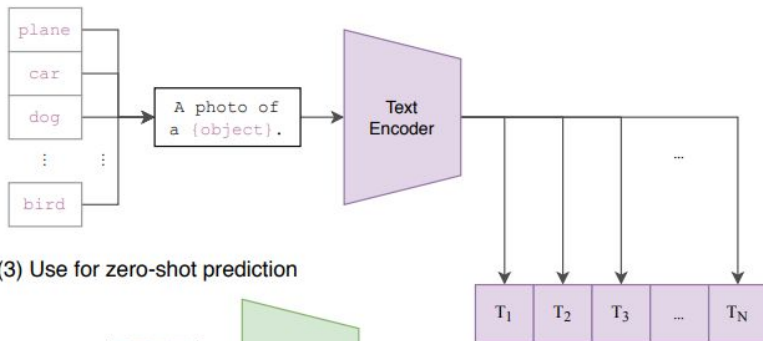
A photo of a {object}. + (with a dog image)

A model is asked to fill a proper object name into the text.

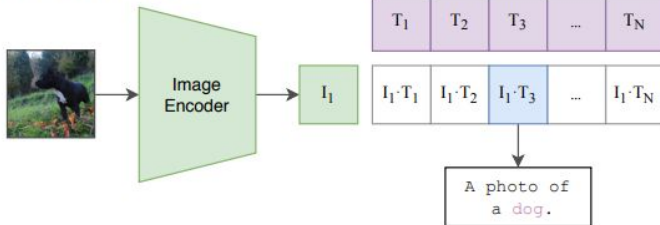
→ A photo of a dog.

CLIP Overview - Inference

(2) Create dataset classifier from label text



(3) Use for zero-shot prediction



- Select a class with the highest similarity with the image, and fill the given sentence (zero-shot)

A photo of a {object}. + (with a dog image)

A model is asked to fill a proper object name into the text.

→ A photo of a dog.

What Is a Vision-Language Model (VLM)?

- Exploit the reasoning and generation capability of LLM to multi-modal models

CLIP

There are {object}s in the image.

There are apples in the image.

VS

VLM

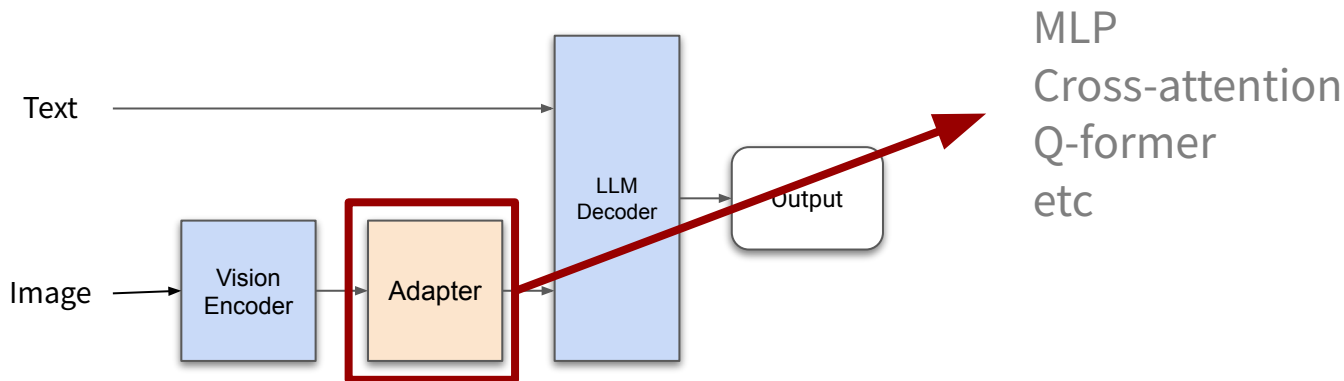
What is in the image?

There are apples in the image,
some are green and others are red.



Aligning Modality Representation

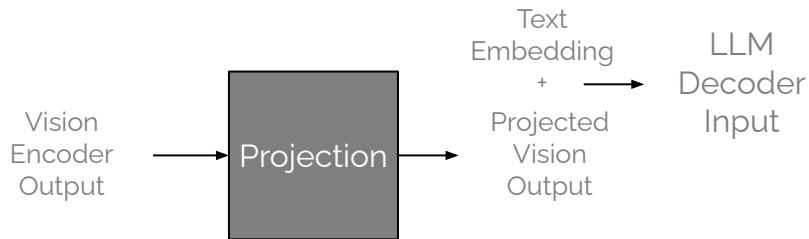
- Project modality embedding space (output representation) to LLM embedding space to make LLM understand various modality representation
- Various implementations of adaptors are proposed



Different Types of Adaptors

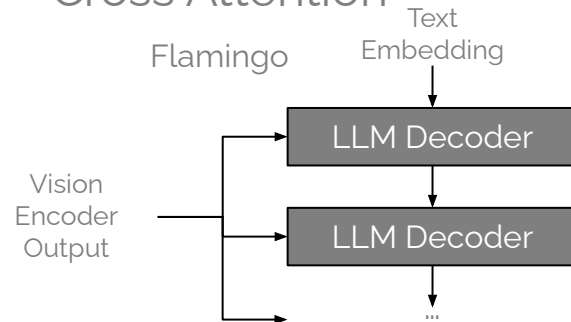
Linear

LLaVA



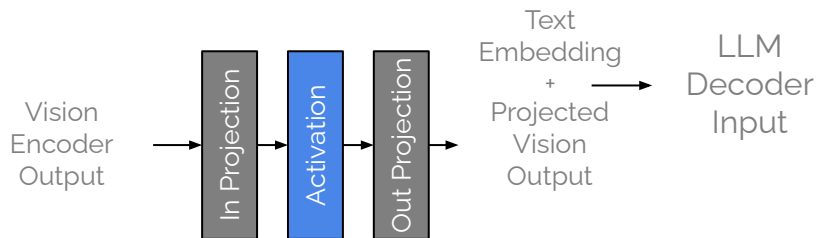
Cross Attention

Flamingo



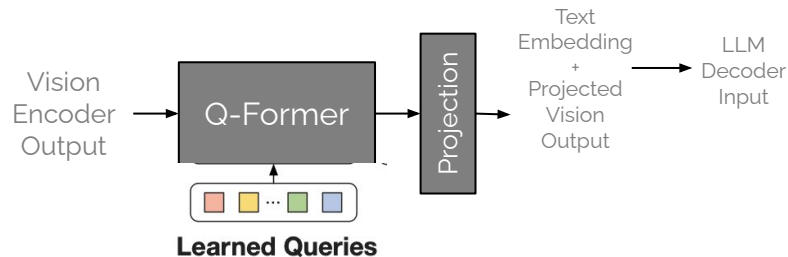
MLP

LLaVA-Next



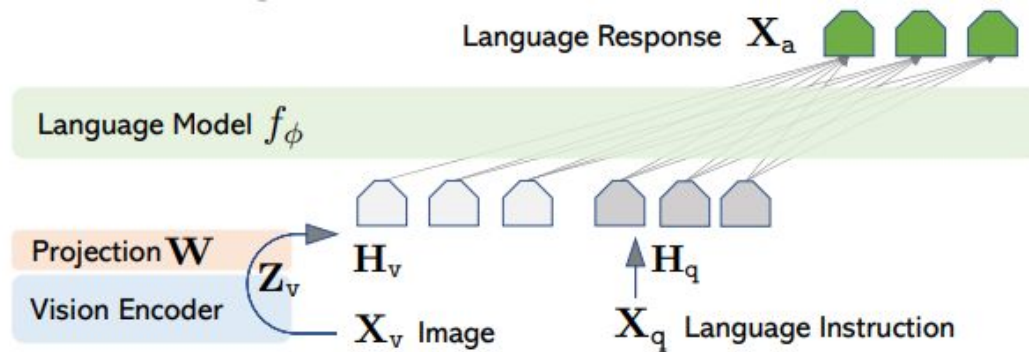
Querying Transformer (Q-former)

BLIP-2



LLaVA

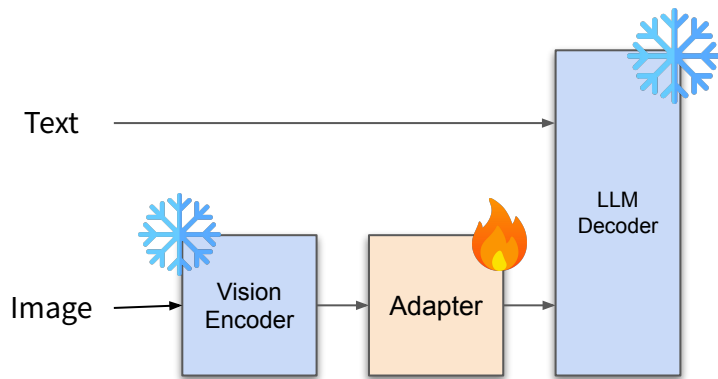
- LLaVA: Large Language and Vision Assistant¹
 - Visual instruction tuning with Instruction-following Data
 - LLaVA Architecture
 - Frozen vision encoder (e.g., CLIP ViT)
 - Frozen language model (e.g., Vicuna)



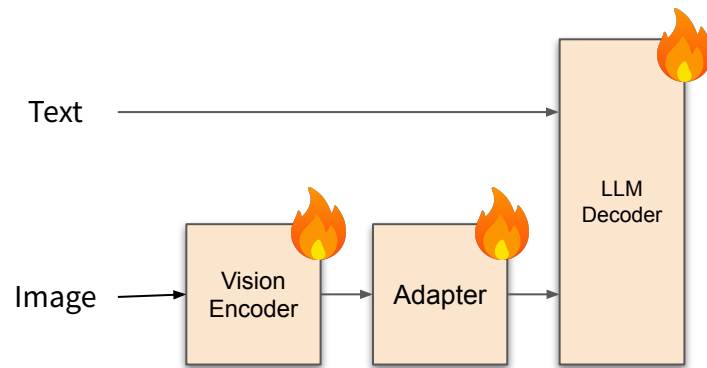
¹Visual Instruction Tuning, 2023

Training VLM

- **Common to use pre-trained vision encoder and LLM**
instead of training entire model from scratch to reduce training cost
- Two-stage training (e.g., LLaVA-Next)
 - a. Pre-train a projector between vision encoder and LLM
 - b. Instruction-tuning the entire model solve unseen problems



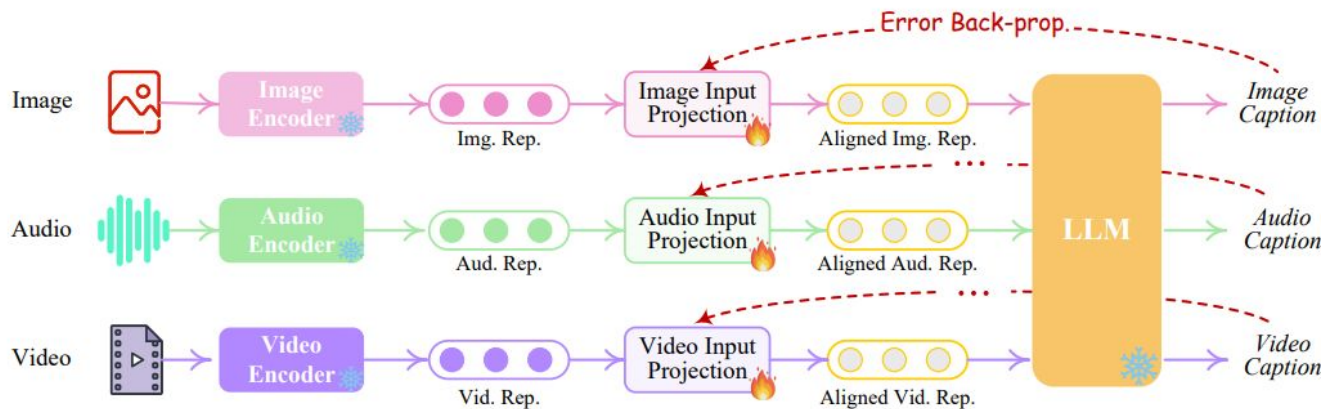
Stage 1: Pre-training a projector



Stage 2: instruction tuning

Towards General Any-to-Text Multimodal LLM

- Encoding-side (understanding)
 - Similar to training VLM: align encoder and LLM embedding space



(a) Encoding-side LLM-centric Alignment

Agenda

- Overview of GenAI
 - From Transformers to LLMs
 - Multi-modal Models (Any-to-Text)
 - **Diffusion Models (Text-to-Any)**
 - LLM AI Agent
- Overview of Systems for GenAI
 - A high-level landscape of systems challenges
- Please ask questions if you don't get something

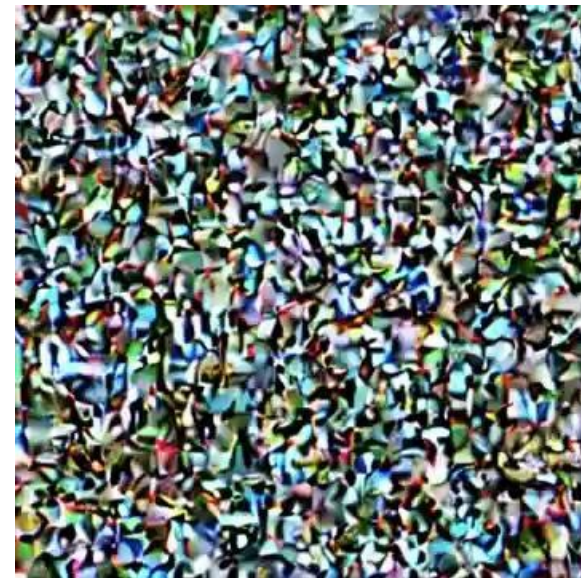
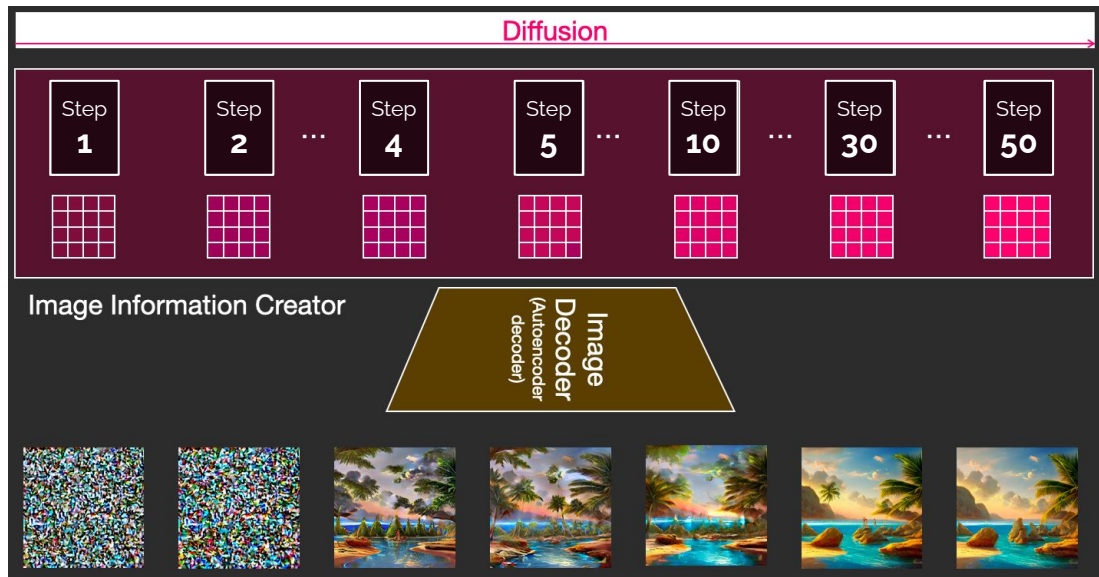
What is a Diffusion Model?

- Use diffusion process to generate continuous data (e.g. image, video, audio)
- Generation Tasks
 - Image - Stable diffusion, Nano banana
 - Video - Sora, AnimateDiff, Zeroscope
 - Audio - Suno, AudioLDM
- Not all generation tasks are done by diffusion



Diffusion Process

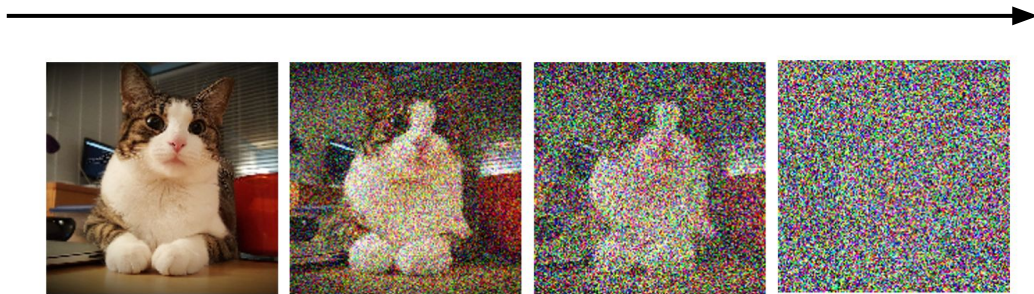
- Start from full random Gaussian noise, **Iteratively denoise** to create an image
- Introduced in Denoising diffusion probabilistic model (DDPM) [\[NeurIPS'20\]](#)



Diffusion Model: Training

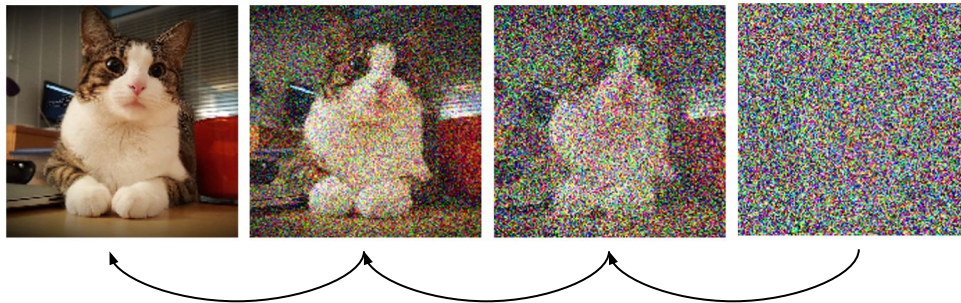
- Forward diffusion process: from original image, gradually add noise to create steps

Forward diffusion process: data to noise



Diffusion Model: Training

- Forward diffusion process: from original image, gradually add noise to create steps
- Reverse diffusion process: Model is trained to **recover the previous step** from the current one by denoising



Backward diffusion process: denoise image with prompt embedding information

Latent Diffusion Model (LDM)

- Original diffusion model (DDPM) **operates in pixel space**
- Require more resources to train and inference
 - Pixel space dimension: $512 \times 512 \times 3 = 786432$
 - Latent dimension for Stable Diffusion: $64 \times 64 \times 3 = 12288$ (64 times smaller!)

Train ADM (Ablated Diffusion Models)

VS

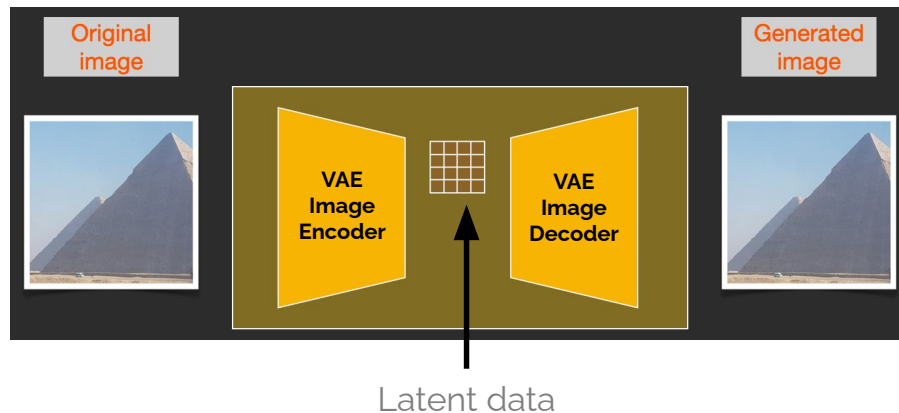
Train LDM

150~1000* V100 GPU days

80~270 V100 GPU days

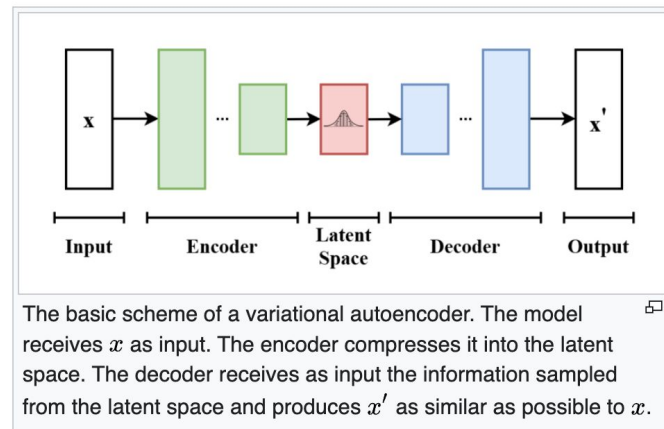
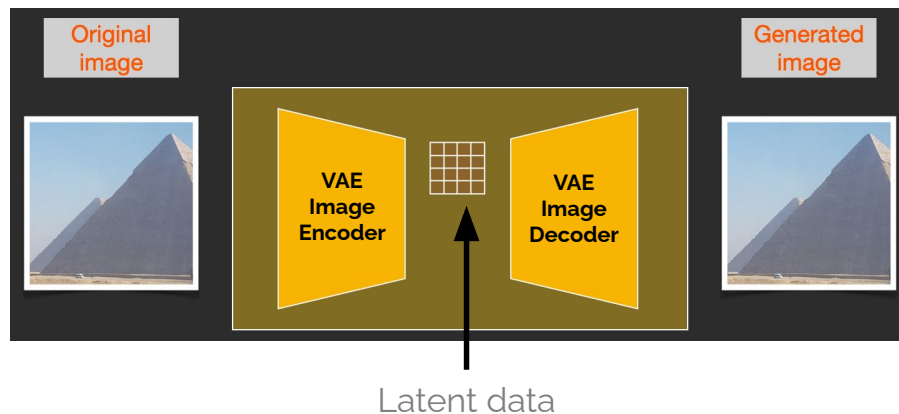
Latent Space and Variational Autoencoder (VAE)

- **Latent Space:** a lower-dimensional representational space, but perceptually equivalent to data (e.g. pixel) space



Latent Space and Variational Autoencoder (VAE)

- **Latent Space: a lower-dimensional representational space**, but perceptually equivalent to data (e.g. pixel) space
- LDM utilizes a model architecture called Variational Autoencoder (VAE) [\[Arxiv'13\]](#)
 - Convert data btw pixel space and latent space using VAE encoder and decoder
 - Train separately

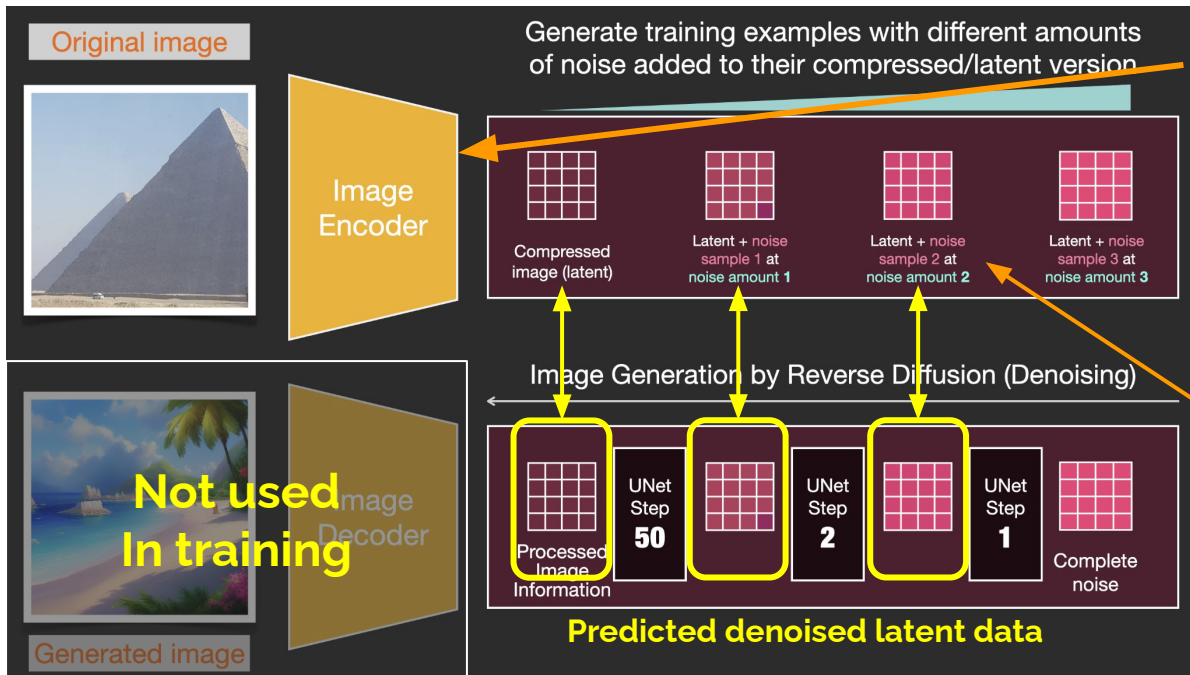


Latent Diffusion Model (LDM): Training

- LDM performs diffusion process in latent space

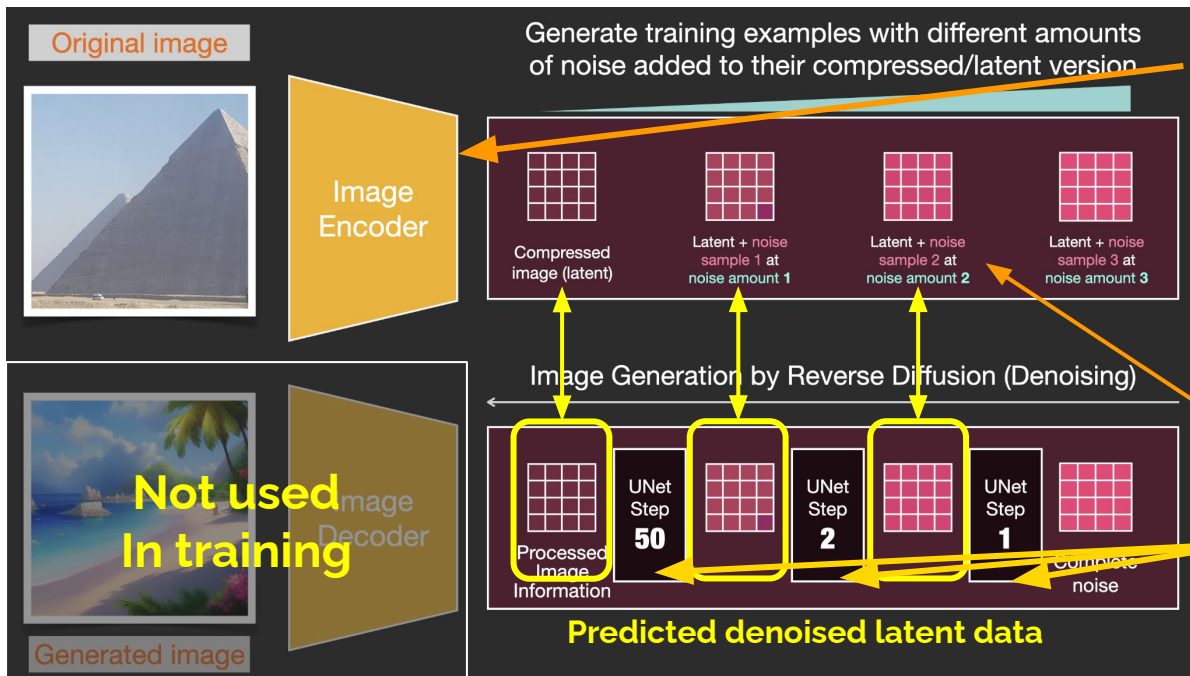
Use VAE image encoder to encode pixels into latent space
Then add noises to generate samples
All samples are in latent space

Model is trained to make predictions be close to **the data samples**



Latent Diffusion Model (LDM): Training

- LDM performs diffusion process in latent space



Use VAE image encoder to encode pixels into latent space
Then add noises to generate samples
All samples are in latent space

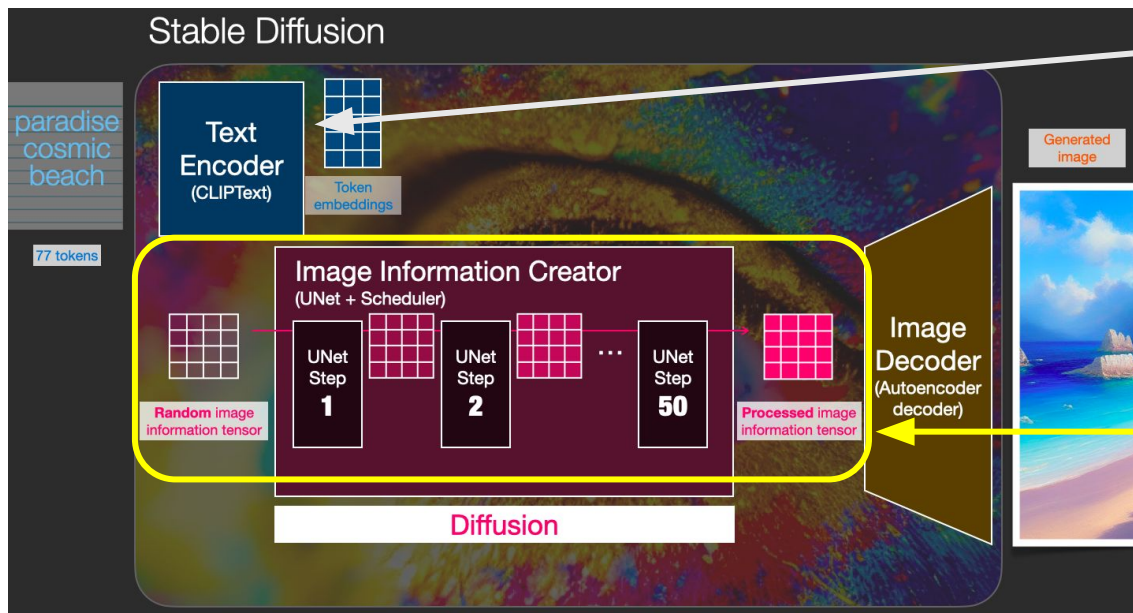
Model is trained to make predictions be close to **the data samples**

Text Encoder (CLIPText)

Text Condition

Latent Diffusion Model (LDM): Inference

- LDM performs diffusion process in latent space



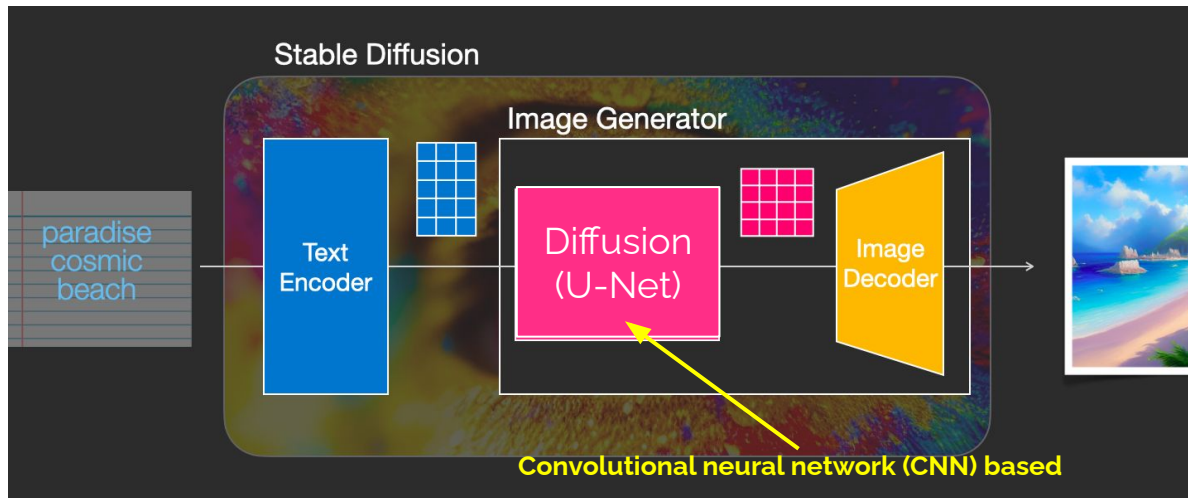
Text encoder is used to encode textual prompt

In text-to-image generation, VAE image encoder is not used as generation starts from random latent noise

Diffusion process is done in latent space, not pixel space

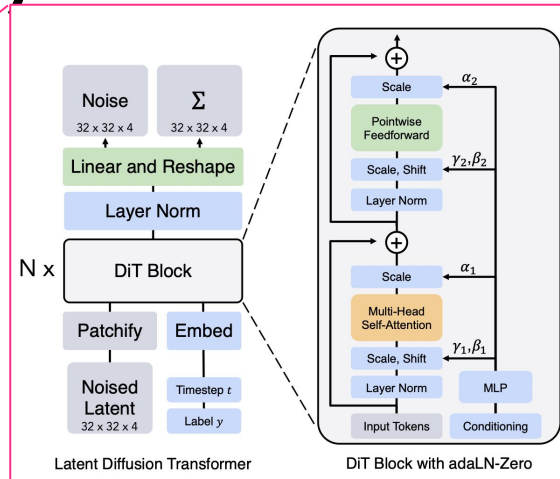
(Latent) Diffusion Transformers (DiT)

- Convolution-based U-Net lacks scalability



[U-Net: Convolutional Networks for Biomedical Image Segmentation \[MICCAI'15\]](#)
[Scalable Diffusion Models with Transformers. \[CVPR23\]](#)

- Higher Gflops, lower FID (better), better scalability



Example of Diffusion Models

OpenAI DALL-E

StabilityAI Stable Diffusion

Midjourney

Stable Diffusion



DALLE 2



Midjourney



Dark alley at night 4k raining aesthetic

Stable Diffusion



DALLE 2



Midjourney



Pyramid shaped mountain above a still lake, covered with snow

Compute Characteristics of Diffusion Models

- Diffusion is really compute intensive although model size is small compared to LLM

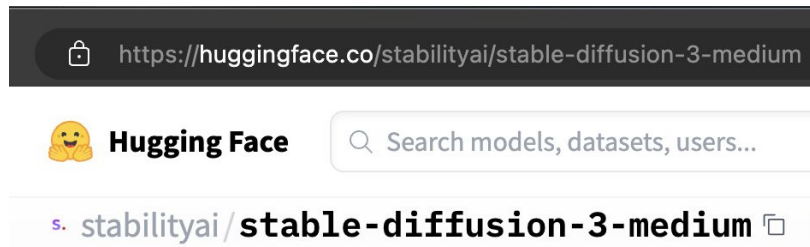
Model	Params	GPU	Throughput
Stable Diffusion XL	2.6B	8xH100	11.83 images/s
Llama2 70b	70B		15875.80 tokens/s
Stable Diffusion XL	2.6B	1xGH200	2.30 images/s
Llama2 70b	70B		4067.52 tokens/s

* Used [20 inference steps](#) per image generation.

Stable Diffusion in HuggingFace

- HF provides a library called *diffusers* for diffusion models train/inference

Model from hub



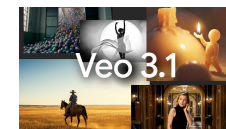
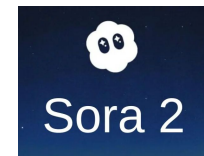
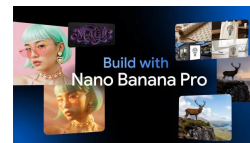
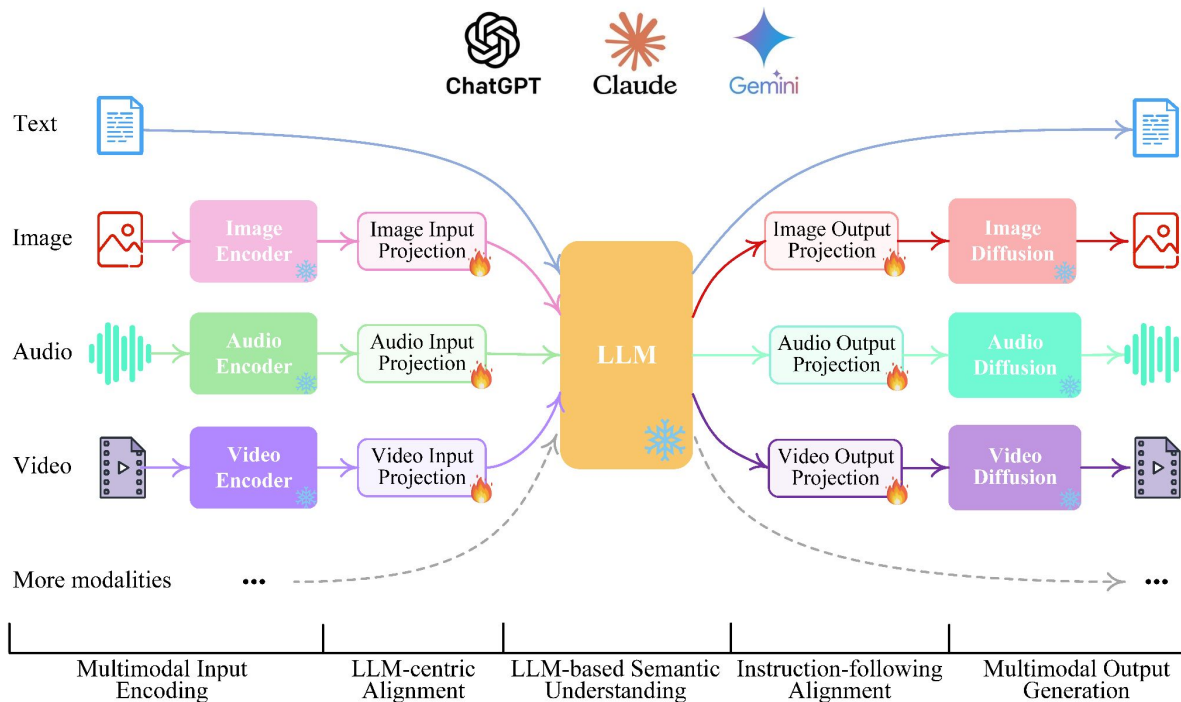
```
import torch
from diffusers import StableDiffusion3Pipeline
```

```
pipe = StableDiffusion3Pipeline.from_pretrained("stabilityai/stable-diffusion-3-medium")
prompt = "a photograph of an astronaut riding a horse"
image = pipe(prompt).images[0]
```

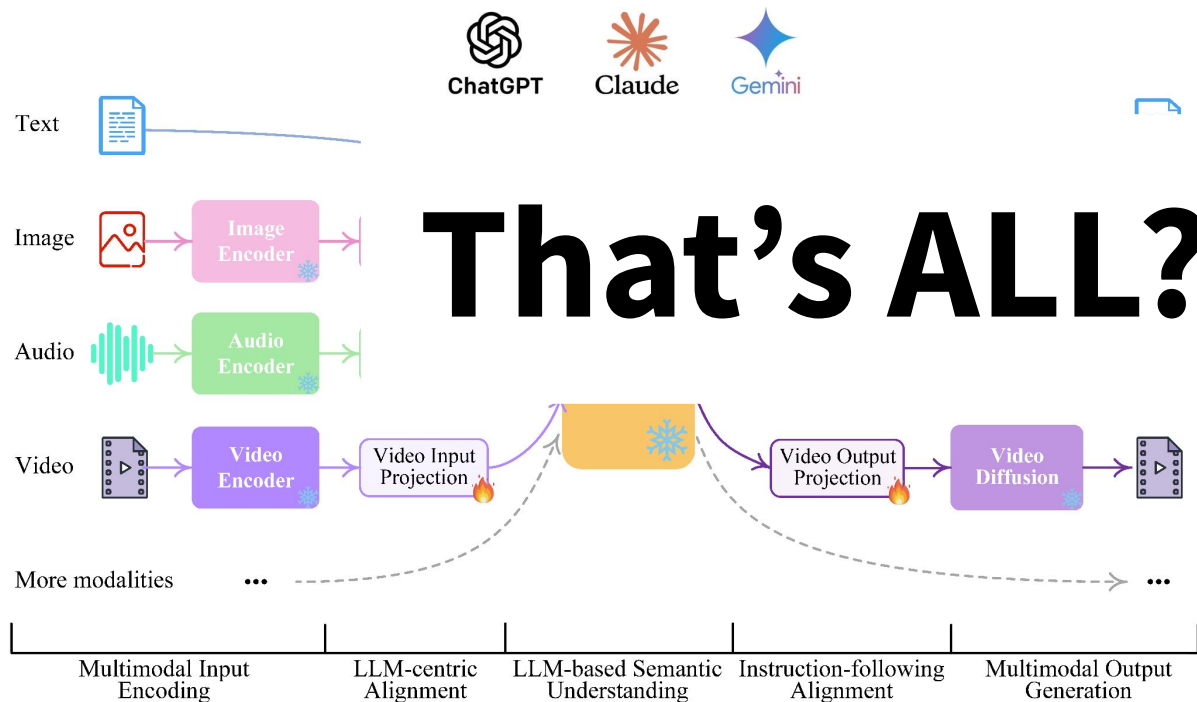
Library



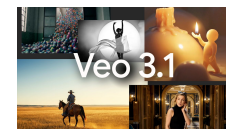
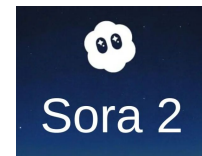
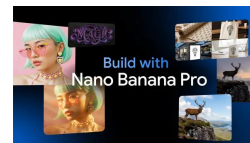
Towards General Any-to-Any Multimodal LLM



Towards General Any-to-Any Multimodal LLM



That's ALL?

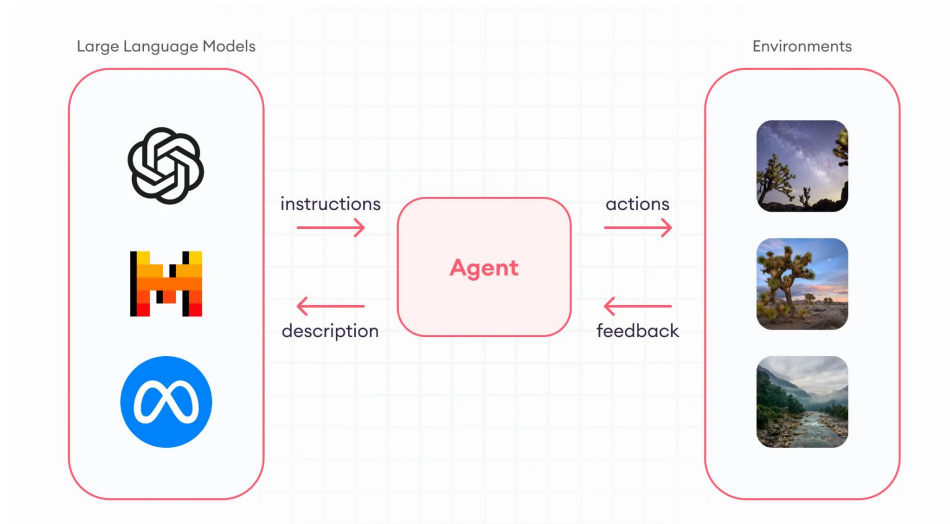


Agenda

- Overview of GenAI
 - From Transformers to LLMs
 - Multi-modal Models (Any-to-Text)
 - Diffusion Models (Text-to-Any)
 - **LLM AI Agent**
- Overview of Systems for GenAI
 - A high-level landscape of systems challenges
- Please ask questions if you don't get something

What are LLM agents?

- Complex tasks need **multi-hop** reasoning and acting
 - Observe current environment
 - Remember past actions
 - Use different tools to adjust their responses based on the situation
- Completing the tasks requires fundamental abilities
 - Reasoning
 - Multimodality handling
 - Tool use proficiency



What can LLM agents do?

- Sample GAIA questions
- Solutions require multiple inference steps

GAIA Question: The attached Excel file contains the sales of menu items for a local fast-food chain. What were the total sales that the chain made from food (not including drinks)? Express your answer in USD with two decimal places.



uploaded.xlsx

GPT4 (Advanced Data Analysis mode):

Working:

```
import pandas as pd

# Load the Excel file
file_path = '/mnt/data/uploaded.xlsx'
df = pd.read_excel(file_path)
```

Working:

```
# Calculate the total sales from food items (not including drinks i.e., Soda)
total_food_sales = df[
    ['Burgers', 'Hot Dogs', 'Salads', 'Fries', 'Ice Cream']
].sum().sum()

# Format the result to USD, with a dollar sign and two decimal places
total_food_sales_formatted = f"${total_food_sales:.2f}"
```

FINAL ANSWER: \$89706.00

Ground truth: 89706.00 ✓

What can LLM agents do?

- **Level 1** questions generally require no tools, or at most one tool but no more than 5 steps.

- Sample GAIA questions
- Solutions require multiple inference steps
- Need access to external tools (e.g., web)

Level 1

Question: What was the actual enrollment count of the clinical trial on *H. pylori* in acne vulgaris patients from Jan-May 2018 as listed on the NIH website?

Ground truth: 90



► Arch Dermatol Res. 2024 Sep 14;316(9):621. doi: [10.1007/s00403-024-03300-w](https://doi.org/10.1007/s00403-024-03300-w) 

Helicobacter pylori and acne vulgaris: is there a relationship?

[Ahmed Abdelfattah Atify](#)^{1*}, [Hanan Mohamed Ahmed Saleh](#)¹, [Abeer Farraq Hussein](#)¹

► Author information ► Article notes ► Copyright and License information

PMCID: PMC11401782 PMID: [39276224](https://pubmed.ncbi.nlm.nih.gov/39276224/)

Patients and methods

Participants

This case-control study included 45 Patients with AV and 45 age and sex matched healthy volunteers as a control group. They were recruited from the Dermatology outpatient clinic, Ain Shams University Hospitals and Teba specialized hospital in Luxor government. The study was done in the period from November 2021 to October 2022. Patients < 12 years and > 40 years, patients with a history of drug intake that affects *H. pylori* such as proton pump inhibitors, clarithromycin, tetracyclines, amoxicillin or metronidazole within the previous 1 month and patients who received any topical or systemic treatment for AV in the past 3 months were excluded. After the approval of Ethics Committee of Faculty of Medicine, Ain Shams University (Approval number: FMASU 670 / 2021), all participants signed an informed consent after explaining for them the objective of the study.

What can LLM agents do?

- Sample GAIA questions
- Solutions require multiple inference steps
- Need access to external tools (e.g., web)
- Answers are unlikely to be found in plain text in training data

- **Level 1** questions generally require no tools, or at most one tool but no more than 5 steps.
- **Level 2** question generally involve more steps, roughly between 5 and 10 and combining different tools is needed.
- **Level 3** are questions for a near perfect general assistant, requiring to take arbitrarily long sequences of actions, use any number of tools, and access to the world in general.

Level 1

Question: What was the actual enrollment count of the clinical trial on *H. pylori* in acne vulgaris patients from Jan-May 2018 as listed on the NIH website?

Ground truth: 90

Level 2



Question: If this whole pint is made up of ice cream, how many percent above or below the US federal standards for butterfat content is it when using the standards as reported by Wikipedia in 2020? Answer as + or - a number rounded to one decimal place.

Ground truth: +4.6

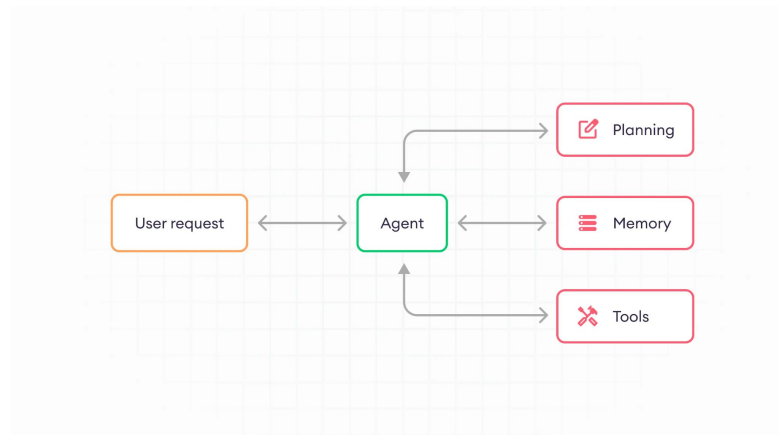
Level 3

Question: In NASA's Astronomy Picture of the Day on 2006 January 21, two astronauts are visible, with one appearing much smaller than the other. As of August 2023, out of the astronauts in the NASA Astronaut Group that the smaller astronaut was a member of, which one spent the least time in space, and how many minutes did he spend in space, rounded to the nearest minute? Exclude any astronauts who did not spend any time in space. Give the last name of the astronaut, separated from the number of minutes by a semicolon. Use commas as thousands separators in the number of minutes.

Ground truth: White; 5876

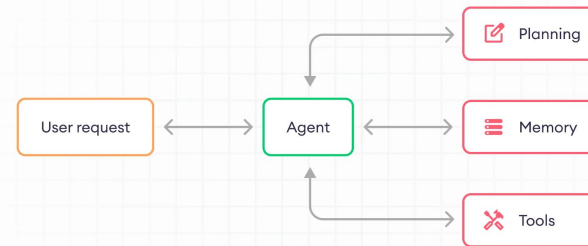
LLM Agent Components

- LLM agents require
 - **Structured plan** - break down complicated tasks into smaller, more manageable parts
 - **Memory to track progress** - short-term memory and long-term memory
 - **Access to necessary tools** - Model Context Protocol (MCP)



LLM Agent Components

- LLM agents require
 - **Structured plan** - break down complicated tasks into smaller, more manageable parts
 - **Memory to track progress** - short-term memory and long-term memory
 - **Access to necessary tools** - Model Context Protocol (MCP)
- Workflow can become quite complex
 - **Single-path** - Chain of Thought (CoT)
 - **Multi-path** - CoT - Self-Consistency (CoT-SC)
 - **Single-hop** - Reasoning and Acting (ReAct)
 - **Multi-hop** - Tree of Thought (ToT)



CoT, Zero-shot CoT



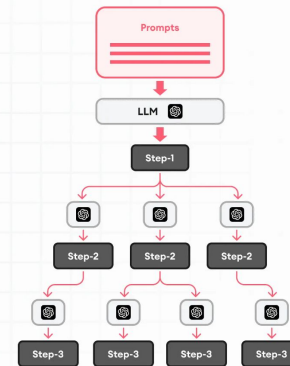
ReWOO, HuggingGPT



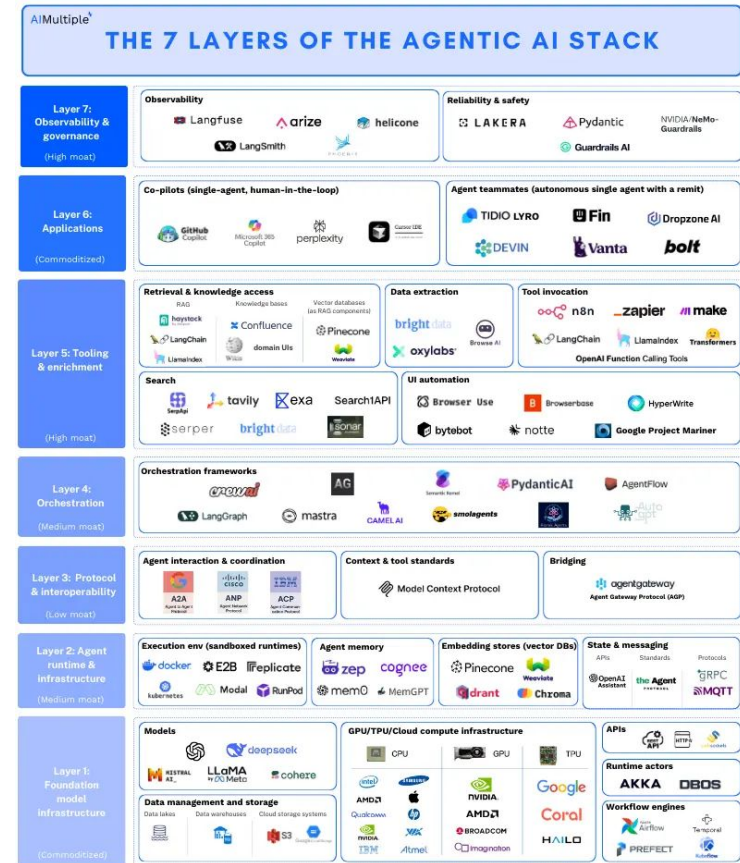
CoT-SC



TOT, LMZSP, RAP



- 75



ReAct Agent Example

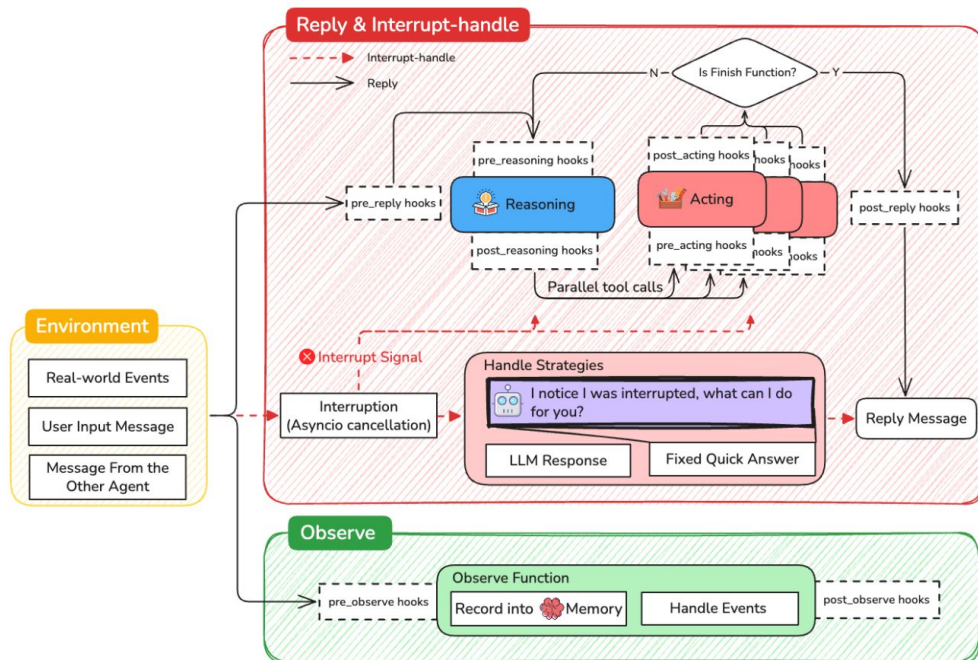
agentscope / src / agentscope / agent / _react_agent.py

Code Blame 543 lines (471 loc) · 18.5 KB

```

54 class ReActAgent(ReActAgentBase):
199     async def reply(
243         # The reasoning-acting loop
244         reply_msg = None
245         for _ in range(self.max_iters):
246             msg_reasoning = await self._reasoning()
247
248             futures = [
249                 self._acting(tool_call)
250                 for tool_call in msg_reasoning.get_content_blocks(
251                     "tool_use",
252                 )
253             ]
254
255             # Parallel tool calls or not
256             if self.parallel_tool_calls:
257                 acting_responses = await asyncio.gather(*futures)
258             else:
259                 # Sequential tool calls
260                 acting_responses = [await _ for _ in futures]
261
262             # Find the first non-None replying message from the acting
263             for acting_msg in acting_responses:
264                 reply_msg = reply_msg or acting_msg
265
266             if reply_msg:
267                 break

```



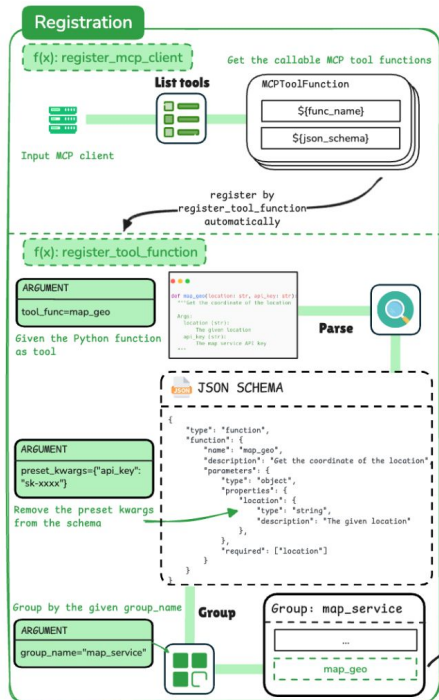


ReAct Agent Example - Reasoning & Acting

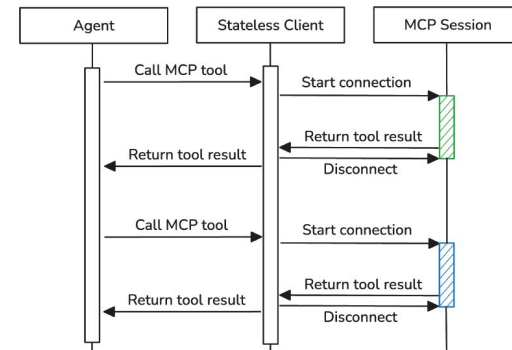
```
54 class ReActAgent(ReActAgentBase):
287     async def _reasoning(
288         self,
289     ) -> Msg:
290         """Perform the reasoning process."""
291         prompt = await self.formatter.format(
292             msgs=[
293                 Msg("system", self.sys_prompt, "system"),
294                 *await self.memory.get_memory(),
295             ],
296         )
297
298         res = await self.model(
299             prompt,
300             tools=self.toolkit.get_json_schemas(),
301         )
302
303         # handle output from the model
304         interrupted_by_user = False
305         msg = None
306         try:
307             if self.model.stream:
308                 msg = Msg(self.name, [], "assistant")
309                 async for content_chunk in res:
310                     msg.content = content_chunk.content
311                     await self.print(msg, False)
312                 await self.print(msg, True)
313             else:
314                 msg = Msg(self.name, list(res.content), "assistant")
315                 await self.print(msg, True)
316
317         return msg
318
319     except asyncio.CancelledError as e:
320         interrupted_by_user = True
321         raise e from None
```

```
54 class ReActAgent(ReActAgentBase):
363     async def _acting(self, tool_call: ToolUseBlock) -> Msg | None:
376         tool_res_msg = Msg(
377             "system",
378             [
379                 ToolResultBlock(
380                     type="tool_result",
381                     id=tool_call["id"],
382                     name=tool_call["name"],
383                     output=[],
384                 ),
385             ],
386             "system",
387         )
388         try:
389             # Execute the tool call
390             tool_res = await self.toolkit.call_tool_function(tool_call)
391
392             response_msg = None
393             # Async generator handling
394             async for chunk in tool_res:
395                 # Turn into a tool result block
396                 tool_res_msg.content[0][ # type: ignore[index]
397                     "output"
398                 ] = chunk.content
399
400             # Skip the printing of the finish function call
401             if (
402                 tool_call["name"] != self.finish_function_name
403                 or tool_call["name"] == self.finish_function_name
404                 and not chunk.metadata.get("success")
405             ):
406                 await self.print(tool_res_msg, chunk.is_last)
407
408             # Return message if generate_response is called successfully
409             if tool_call[
410                 "name"
411             ] == self.finish_function_name and chunk.metadata.get(
412                 "success",
413                 True,
414             ):
415                 response_msg = chunk.metadata.get("response_msg")
```

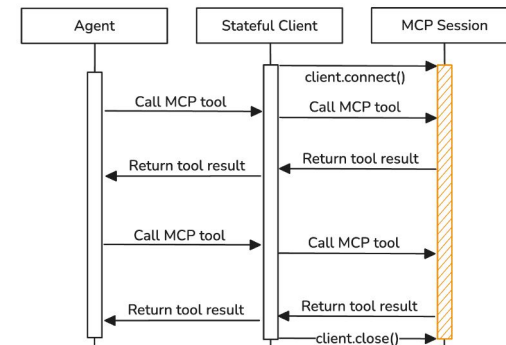
ReAct Agent Example - MCP ToolKit



```
1  {
2
3  "type": "function",
4  "function": {
5    "name": "browser_close",
6    "description": "Close the page",
7    "parameters": {
8      "type": "object",
9      "properties": {},
10     "required": []
11   }
12 },
13
14
15 "type": "function",
16 "function": {
17   "name": "browser_resize",
18   "description": "Resize the browser window",
19   "parameters": {
20     "type": "object",
21     "properties": {
22       "width": {
23         "type": "number",
24         "description": "Width of the browser window"
25       },
26       "height": {
27         "type": "number",
28         "description": "Height of the browser window"
29       }
30     },
31     "required": [
32       "width",
33       "height"
34     ]
35   }
36 },
37
38 "type": "function",
39 "function": {
40
```



(a) Stateless MCP client



(b) Stateful MCP client

Single-agent Conversation

```
1 import asyncio, os
2
3 from agentscope.agent import ReActAgent, UserAgent
4 from agentscope.formatter import DashScopeChatFormatter
5 from agentscope.memory import InMemoryMemory
6 from agentscope.model import DashScopeChatModel
7 from agentscope.tool import Toolkit, execute_shell_command, execute_python_code
8     ↪ , view_text_file
9
10 async def main() -> None:
11     """The main entry point for the ReAct agent example."""
12     toolkit = Toolkit()
13     toolkit.register_tool_function(execute_shell_command)
14     toolkit.register_tool_function(execute_python_code)
15     toolkit.register_tool_function(view_text_file)
16
17     agent = ReActAgent(
18         name="Friday",
19         sys_prompt="You are a helpful assistant named Friday.",
20         model=DashScopeChatModel(
21             api_key=os.environ.get("DASHSCOPE_API_KEY"),
22             model_name="qwen-max",
23             enable_thinking=False,
24             stream=True,
25         ),
26         formatter=DashScopeChatFormatter(),
27         toolkit=toolkit,
28         memory=InMemoryMemory(),
29         # Additional arguments setting
30         long_term_memory=MemOLongTermMemory(),
31         long_term_memory_mode="both",
32         parallel_tool_call=True,
33     )
34     user = UserAgent("Bob")
35
36     msg = None
37     while True:
38         msg = await user(msg)
39         if msg.get_text_content() == "exit":
40             break
41         msg = await agent(msg)
42
43
44 asyncio.run(main())
```

Multi-agent Conversation

```

1 import asyncio, os
2
3 from agentscope.agent import ReActAgent, UserAgent
4 from agentscope.formatter import DashScopeChatFormatter
5 from agentscope.memory import InMemoryMemory
6 from agentscope.model import DashScopeChatModel
7 from agentscope.tool import Toolkit, execute_shell_command, execute_python_code
8     ↪ , view_text_file
9
10 async def main() -> None:
11     """The main entry point for the ReAct agent example."""
12     toolkit = Toolkit()
13     toolkit.register_tool_function(execute_shell_command)
14     toolkit.register_tool_function(execute_python_code)
15     toolkit.register_tool_function(view_text_file)
16
17     agent = ReActAgent(
18         name="Friday",
19         sys_prompt="You are a helpful assistant named Friday.",
20         model=DashScopeChatModel(
21             api_key=os.environ.get("DASHSCOPE_API_KEY"),
22             model_name="qwen-max",
23             enable_thinking=False,
24             stream=True,
25         ),
26         formatter=DashScopeChatFormatter(),
27         toolkit=toolkit,
28         memory=InMemoryMemory(),
29         # Additional arguments setting
30         long_term_memory=MemOLongTermMemory(),
31         long_term_memory_mode="both",
32         parallel_tool_call=True,
33     )
34     user = UserAgent("Bob")
35
36     msg = None
37     while True:
38         msg = await user(msg)
39         if msg.get_text_content() == "exit":
40             break
41         msg = await agent(msg)
42
43 asyncio.run(main())

```

```

1 import asyncio, os
2
3 from agentscope.agent import ReActAgent
4 from agentscope.formatter import DashScopeMultiAgentFormatter
5 from agentscope.message import Msg
6 from agentscope.model import DashScopeChatModel
7 from agentscope.pipeline import MsgHub, sequential_pipeline
8
9
10 def create_agent(name: str, age: int, career: str, character: str):
11     """Create a participant agent with a specific name, age, and character."""
12     return ReActAgent(
13         name=name,
14         sys_prompt=(
15             f"You're a {age}-year-old {career} named {name} and you're "
16             f"a {character} person."
17         ),
18         model=DashScopeChatModel(
19             model_name="qwen-max",
20             api_key=os.environ["DASHSCOPE_API_KEY"],
21             stream=True,
22         ),
23         # Use multiagent formatter because multiple entities involves
24         formatter=DashScopeMultiAgentFormatter(),
25     )
26
27 async def main() -> None:
28     """Run a multi-agent conversation workflow."""
29     # Create multiple participant agents with different characteristics
30     alice = create_agent("Alice", 30, "teacher", "friendly")
31     bob = create_agent("Bob", 14, "student", "rebellious")
32     charlie = create_agent("Charlie", 28, "doctor", "thoughtful")
33
34     # Create a conversation where participants introduce themselves
35     async with MsgHub(
36         participants=[alice, bob, charlie],
37         # The greeting message will be sent to all participants at the start
38         announcement=Msg(
39             "system",
40             "Now you meet each other with a brief self-introduction.",
41             "system",
42         ),
43     ) as hub:
44         # Quick construct a pipeline to run the conversation
45         await sequential_pipeline([alice, bob, charlie])
46
47         # Delete a participant agent from the hub and fake a broadcast message
48         hub.delete(bob)
49         await hub.broadcast(
50             Msg(
51                 "bob",
52                 "I have to start my homework now, see you later!",
53                 "assistant",
54             ),
55         ),
56     )
57     await alice()
58     await charlie()
59
60 asyncio.run(main())

```

Agenda

- Overview of GenAI
 - From Transformers to LLMs
 - Multi-modal Models (Any-to-Text)
 - Diffusion Models (Text-to-Any)
 - LLM AI Agent
- **Overview of Systems for GenAI**
 - **A high-level landscape of systems challenges**
- Please ask questions if you don't get something

Recap

- Transformers
 - Training on next token prediction and doing autoregressive text generation.
 - It works pretty well!
- Scaling laws
 - More parameters and more data shall make your model even better.
 - Scale up more and more! We haven't seen it significantly slow down yet.
- Which brought us ...

Unprecedented scale

- Unprecedented scale of adoption
 - ChatGPT passed 1 million users in 5 days and has 800 million weekly active users now.
 - Why? Transformers worked pretty well.
- Unprecedented scale of ML workloads
 - Llama 4 series pre-training took 7.4 million GPU hours. OpenAI GPT OSS 2.1 million.
 - One pre-training job can use 100k GPUs, and there are plans to scale further.
 - We don't know how many GPUs are serving ChatGPT, but it's probably a fair amount.
 - Why? Scaling laws fueled the fad, and it has not betrayed us yet.

Unprecedented scale

**With great scale
comes great systems challenges**

Pre-training

- Much compute and memory requirements
- Much GPU
 - Like, 100k or more for a single training run
 - We are already in the era of multi-datacenter training
 - How are we going to split one model training into 100k+ GPUs?
- Much communication
 - The 100k GPUs need to talk to each other
 - If not optimized well, expensive GPUs are going to spend all their lives doing communication
- Much power and energy
 - An H100 GPU draws anywhere between 100 to 700 W during training, and we have 100k of it
 - Building power plants take years but we need that power now
- Much failure
 - Say a GPU fails at any moment with probability 0.01%
 - The probability of no GPUs failing is $(1 - 0.0001)^{100000} = 0.00004538$

Post-training

- All the problems of pre-training
- Unprecedented adoption also brought about so many requirements
 - Be friendly
 - Be helpful
 - Don't hallucinate
 - Do reasoning to solve challenging problems (but also don't take forever)
 - Understand multimodal (image, video, audio) inputs
 - Generate multimodal content, too
- After pre-training, there are typically **tens** of post-training phases with
 - Supervised fine-tuning (SFT)
 - Reinforcement learning (RL)
 - Different environments and rewards
 - Not all rewards are trivial to define

Inference serving

- 1% gain amounts to a lot if you're serving ChatGPT
 - Every drop of efficiency gain is precious!
- LLM text generation has a very distinct computation characteristic
 - Generating the first output token: computation ops > memory ops (**compute-bound**)
 - All other output tokens: computation ops < memory ops (**memory-bound**)
 - Memory management is key!
 - Multi-model model text generation balance different modalities
- Diffusion models are mostly compute-bound
- Many use cases and application requirements
 - Conversational
 - Agents - workflow, control plane and data plane
- ⇒ More and more complex and heterogeneous model architectures

Infrastructure

- Keeping a datacenter with $O(10,000)$ GPUs in it healthy and optimized is hard
- Power management
 - Getting power is the first grand challenge
 - Power management at datacenter-scale is challenging (e.g., mitigating fluctuations)
- Datacenter networking
 - Large amounts of bursty communication during training
 - One straggler bottlenecks the whole training job
- Hardware heterogeneity
 - Especially accelerators (GPUs, TPUs, MTIAs, etc.)
 - (1) You can't get enough of one type, and (2) relying on a single type introduces supply risks
- Multi-datacenter training makes all of the above even harder

Advanced Scalable Systems

- ML advances → Scalability challenges for systems
- We manage to build systems that scale → Accelerates ML advances
- It's a great time to be alive and building!